

目次

本書について	6
本書の特徴とおすすめポイント	6
前提条件	6
すぐに開発を始めたい方へ	7
CHIRIMEN とは	8
CHIRIMEN のメリット	8
コミュニティについて	9
入門用	10
hello-real-world	10
Lチカ (GPIO OUTPUT)	10
ギヤードモーター part1	12
ギヤードモーター part2	14
ギヤードモーター part3	15
ギヤードモーター part4	16
ギヤードモーター part5	17
MOSFET P718D モジュール	18
MOSFET D4184 モジュール	20
GPIO 関連	22
基本的な GPIO 操作	22
GPIO タクトスイッチ	22
GPIO スイッチと LED	24
GPIO スイッチとギヤードモーター	26
タッチセンサー	28
GPIO タッチセンサー	28
カメラ	30
GPIO スイッチによるカメラ撮影 (GPIO-Camera)	30

環境センサー	37
温度・湿度センサー	37
AHT10 温湿度センサー	39
AHT20 温湿度センサー	41
HTU21D 温湿度センサー	43
SHT30 温湿度センサー	45
SHT40 温湿度センサー	47
気圧センサー	49
BMP180 大気圧・温度センサー	49
BMP280 温度・気圧センサー	51
BME280 温度・湿度・気圧センサー	53
空気品質センサー	55
ENS160 空気質センサー	55
SGP40 ガスセンサー	58
CO2 センサー	60
SCD40 CO2 センサー	60
光センサー	62
BH1750 光センサー	62
LTR390 UV センサー	64
TSL2561 Grove Digital Light Sensor 光センサー	66
TSL2591 照度センサー	68
VEML6070 UV センサー	70
色センサー	72
TCS34725 RGB カラーセンサー	72
S11059 デジタルカラーセンサー	74

モーション・位置センサー	76
加速度・ジャイロセンサー	76
ADXL345 3 軸加速度センサー	76
ICM20948 9 軸センサー	78
MPU6050 3 軸ジャイロ 3 軸加速度 複合センサー	80
MPU9250 3 軸ジャイロ 3 軸加速度 3 軸 磁気複合セン サー	82
距離センサー	84
GP2Y0E03 測距センサー 40 mm - 0.1 m	84
VL53L0X レーザー測距センサー 30 mm - 2 m	86
VL53L1X レーザー距離センサー	88
角度センサー	90
AS5600 磁気式角度センサー	90
ジェスチャーセンサー	93
PAJ7620 Grove Gesture ジェスチャー認識センサー ..	93
雷センサー	95
AS3935 雷センサー	95
ディスプレイ・LED	99
OLED ディスプレイ	99
SSD1306 OLED ディスプレイ	99
SSD1308 Grove OLED ディスプレイ	101
LED ドライバー	103
HT16K33 LED マトリクスドライバ	103
NeoPixel	109
Neopixel LED	109
電子ペーパー	114
WAVESHARE20471 複合センサーボード	114

モーター制御	118
ステッピングモータードライバー	118
A4988 ドライバによるステッピングモータ制御	118
H ブリッジ	121
HBridge1	121
PCA9685 16 チャンネル PWM ドライバ.....	124
PWM コントローラー	127
PCA9685 16 チャンネルサーボモーター PWM ドライバー	127
アナログ・デジタル変換.....	129
ADC	129
ADS1015 12 ビット AD コンバータ	129
ADS1115 16bit ADC 差動入力によるロードセルの使用	131
ADS1115 16bit ADC	133
PCF8591 8bit AD,DA コンバーター	135
電流センサー	137
INA219 電流センサー	137
オーディオ	140
オーディオプレーヤー	140
DFPlayer Mini	140
音声録音	143
ISD1820 ボイスレコーダ (GPIO OUTPUT)	143

その他.....	145
GPS.....	145
Serial GPS	145
サーマルカメラ	148
AMG8833 サーモグラフィー.....	148
赤外線温度センサー	150
MLX90614 赤外線温度センサー	150
タッチセンサー	152
MPR121 静電容量センサ(12ch)	152

本書について

本書は、Node.js 上で WebGPIO や WebI2C を使って CHIRIMEN 対応デバイスを制御するためのレシピ集です。

デバイスの概要、配線、ドライバ導入、動作確認コードを順に掲載しています。実機を使いながら、Node.js によるデバイス制御を体験できます。

本書の特徴とおすすめポイント

- 各センサーごとに、わかりやすい回路図を掲載
- コピー＆ペーストですぐ使える動作確認済みコードを収録
- コミュニティで検証された信頼性の高い部品を紹介

前提条件

本書の内容を試すには、以下の環境を推奨。

- デバイス：Raspberry Pi Zero W（Wi-Fi 搭載モデル）
- 主な購入先
 - KSY：Raspberry Pi Zero WH [RASPIZWHSC0065]^{*1}
 - SWITCH SCIENCE：Raspberry Pi Zero WH [RPI-ZERO-WH]^{*2}
- OS：CHIRIMEN Lite^{*3}（Raspbian ベース）
 - ダウンロード^{*4}
- Node.js のバージョン^{*5}
 - v22.x 以上を推奨

1. <https://x.gd/HytS7>

2. <https://x.gd/KF2aF>

3. <https://github.com/chirimen-oh/chirimen-lite>

4. <https://github.com/chirimen-oh/chirimen-lite/releases>

5. <https://nodejs.org/ja/about/previous-releases>

- 必要なライブラリ
 - I²C 制御用ライブラリ： node-web-i2c^{*6}
 - GPIO 制御用ライブラリ： node-web-gpio^{*7}

すぐに開発を始めたい方へ

まずは以下の公式資料を参照又は購入してください

- 公式サイト：CHIRIMEN Raspberry Pi Zero W チュートリアル^{*8}
- 書籍：ちりめんぱいぜろちゅーとりある^{*9}

6. <https://www.npmjs.com/package/node-web-i2c>

7. <https://www.npmjs.com/package/node-web-gpio>

8. <https://tutorial.chirimen.org/pizero/>

9. <https://x.gd/OmMZJ>

CHIRIMEN とは

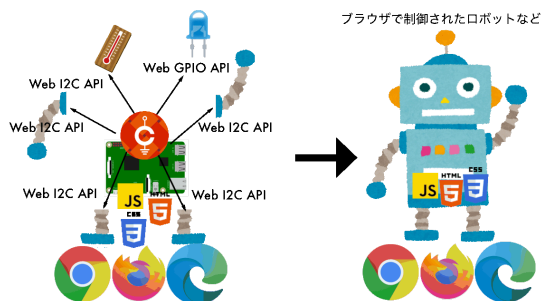


図 1: CHIRIMEN_pf

CHIRIMEN は、JavaScript や HTML といった Web 技術を用いて、センサーやモーターなどの電子部品を簡単に制御できるプロトタイピング環境です。

Raspberry Pi に WebGPIO API^{*1} や WebI2C API^{*2} を組み合わせることで、ブラウザ上からハードウェアを直接制御できます。

教育や試作開発を目的とし、誰でも手軽に扱えるように設計された、オープンで自由な環境です。

CHIRIMEN のメリット

- 標準技術ベース：学習のハードルが低く、スキルの汎用性が高い
- Web 技術との親和性：UIやサービス連携が容易
- 開発しやすさ：ブラウザ上で実行・開発可能
- 情報の検索性：ネット上の情報が豊富で調べやすい

1. <http://browserobo.github.io/WebGPIO>

2. <http://browserobo.github.io/WebI2C>

コミュニティについて

不明点や質問がある場合は、CHIRIMEN コミュニティまでお気軽にご相談ください

- Slack^{*3}
- Github^{*4}
- X^{*5}
- Facebook^{*6}

3. <http://chirimen-oh.slack.com/>

4. <https://github.com/chirimen-oh/>

5. https://x.com/chirimen_oh

6. <https://www.facebook.com/groups/chirimen/>

入門用

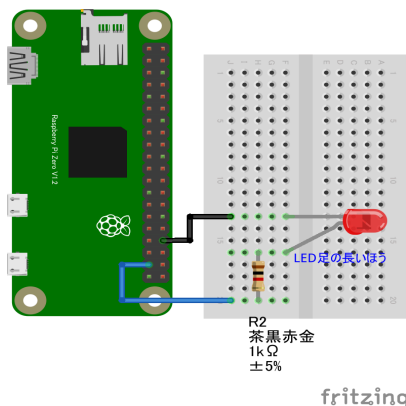
hello-real-world

Lチカ (GPIO OUTPUT)

概要

- LED を点滅させます。

配線図



GPIO ポート 26 に LED と抵抗を直列に接続します。

CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

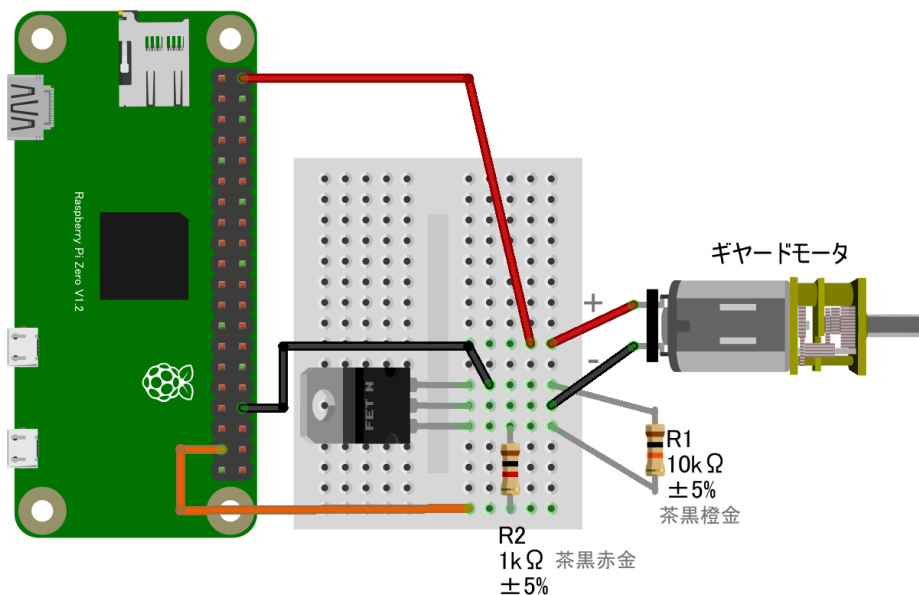
```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";  
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));  
  
async function blink() {  
  const gpioAccess = await requestGPIOAccess();  
  const port = gpioAccess.ports.get(26);  
  
  await port.export("out");  
  
  for (;;) {  
    await port.write(1);  
    await sleep(1000);  
    await port.write(0);  
    await sleep(1000);  
  }  
}  
  
blink();
```

ギヤードモーター part1

概要

- ギヤードモーターの回転・停止

配線図



fritzing

GPIO ポート 26 にモーター制御回路を繋ぎます。※このセクションのコードは「Lチカ」と共通のため、配線のみ異なる点にご注意ください。

CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

async function blink() {
  const gpioAccess = await requestGPIOAccess();
  const port = gpioAccess.ports.get(26);

  await port.export("out");

  for (;;) {
    await port.write(1);
    await sleep(1000);
    await port.write(0);
    await sleep(1000);
  }
}

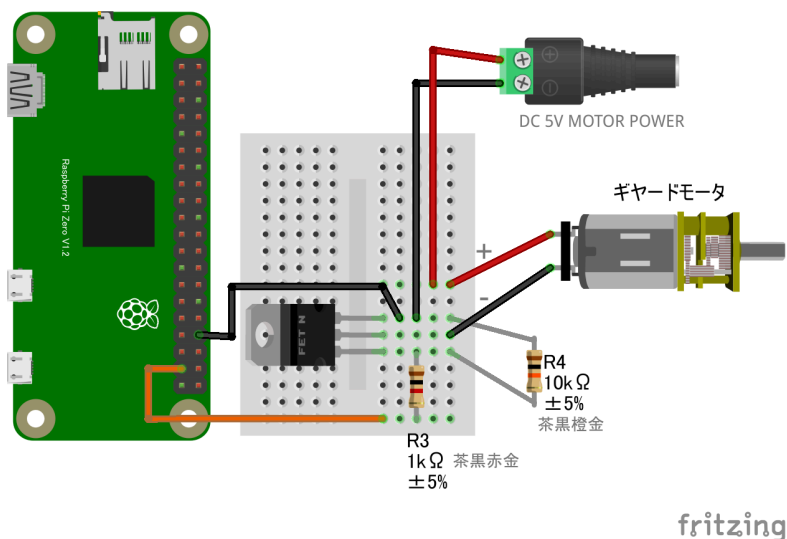
blink();
```

ギヤードモーター part2

概要

- ギヤードモーター (GPIO OUTPUT)外部電源を使用する場合
- ギヤードモーターの回転・停止

配線図



5V 以外の DC 電源も使用可能

CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

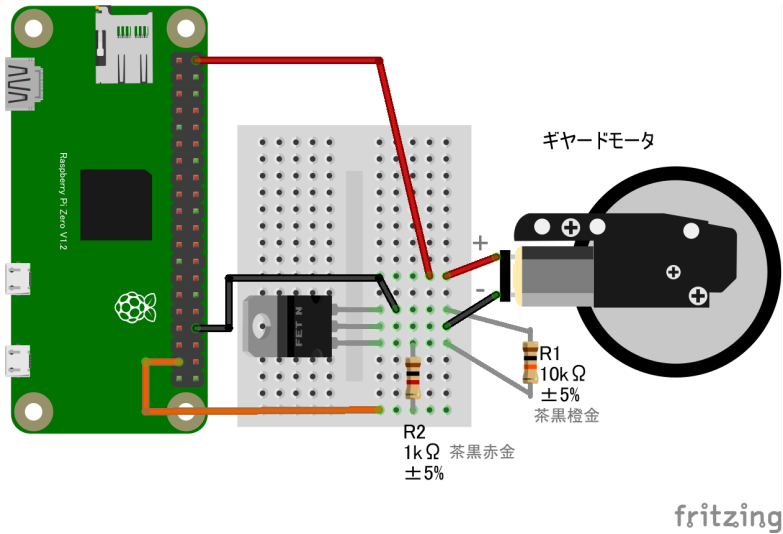
- ギヤードモーター part1 と同じコードです。

ギヤードモーター part3

概要

- ギヤードモーターの回転・停止

配線図



CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

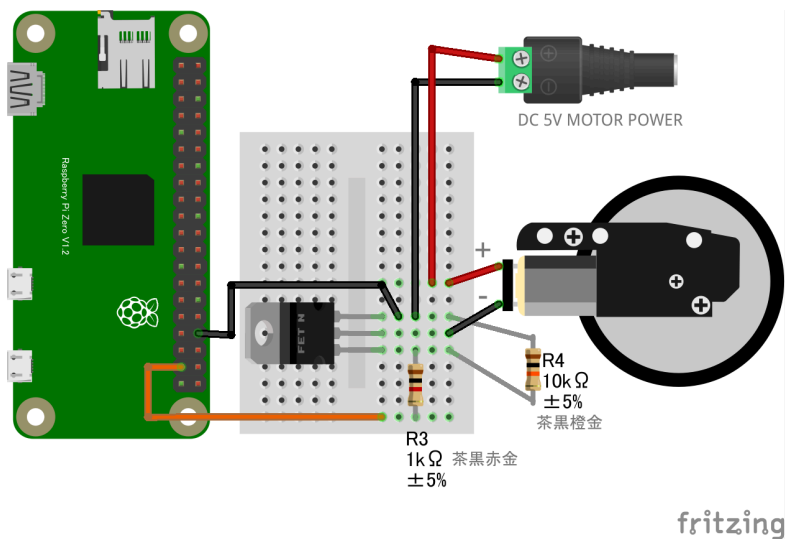
- ギヤードモーター part1 と同じコードです。

ギヤードモーター part4

概要

- ギヤードモーターの回転・停止

配線図



CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

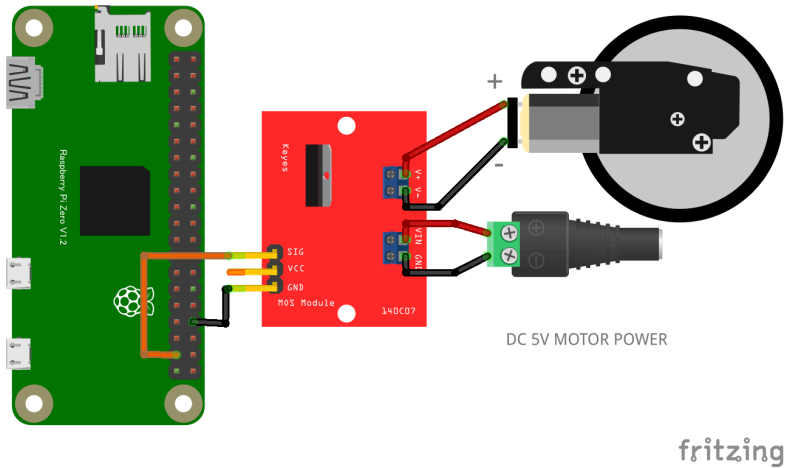
- ギヤードモーター part1 と同じコードです。

ギヤードモーター part5

概要

- ギヤードモーターの回転・停止

配線図



CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

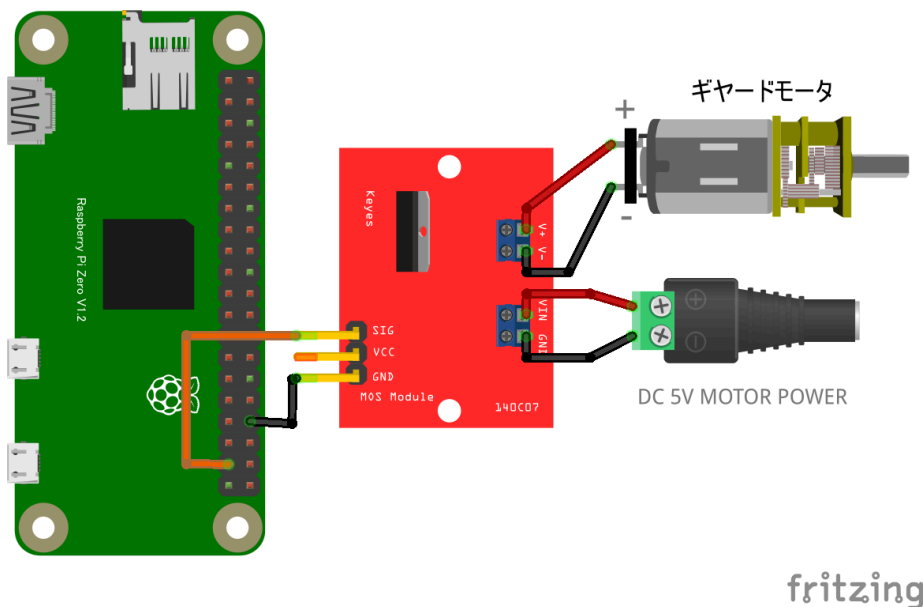
- ギヤードモーター part1 と同じコードです。

MOSFET P718Dモジュール

概要

- MOSFET モジュール基板を使用する場合 P718D モジュール

配線図



CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

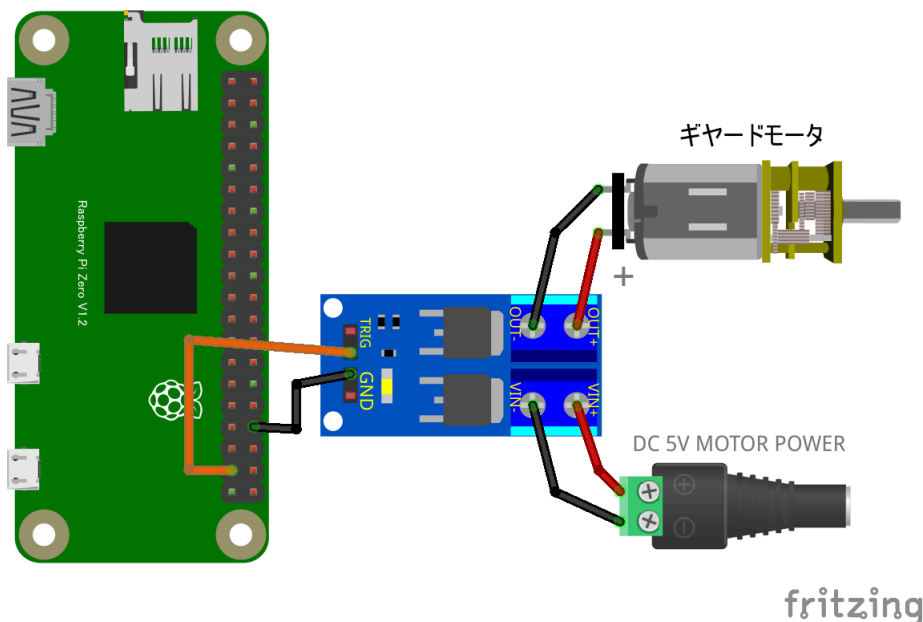
```
async function blink() {  
  const gpioAccess = await requestGPIOAccess();  
  const port = gpioAccess.ports.get(26);  
  
  await port.export("out");  
  
  for (;;) {  
    await port.write(1);  
    await sleep(1000);  
    await port.write(0);  
    await sleep(1000);  
  }  
}  
  
blink();
```

MOSFET D4184 モジュール

概要

- MOSFET モジュール基板を使用する場合 D4184 モジュール

配線図



5V 以外の DC 電源も使用可能です。

CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";  
const sleep = msec => new Promise(resolve => setTimeout(reso
```



```
lve, msec));

async function blink() {
  const gpioAccess = await requestGPIOAccess();
  const port = gpioAccess.ports.get(26);

  await port.export("out");

  for (;;) {
    await port.write(1);
    await sleep(1000);
    await port.write(0);
    await sleep(1000);
  }
}

blink();
```

GPIO 関連

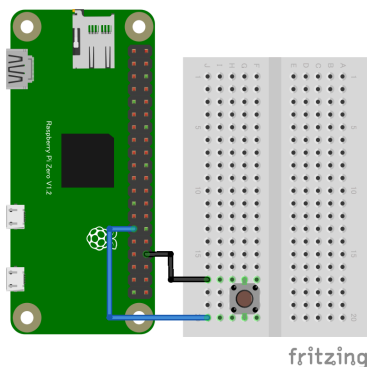
基本的な GPIO 操作

GPIO タクトスイッチ

概要

- GPIO に接続されたタクトスイッチの状態（ON/OFF）を監視、ログ出力

配線図



CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

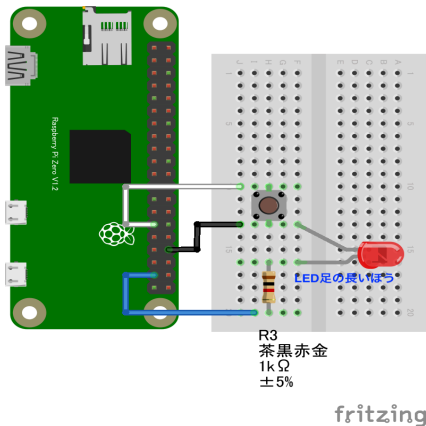
```
async function switchCheck() {  
  const gpioAccess = await requestGPIOAccess();  
  const port = gpioAccess.ports.get(5);  
  
  await port.export("in");  
  
  for(;;){  
    const v = await port.read();  
    console.log(v);  
    await sleep(300);  
  }  
}  
  
switchCheck();
```

GPIO スイッチと LED

概要

- スイッチを押すと LED を点灯、離すと消灯させる基本的な動作です。

配線図



GPIO ポート 5 にスイッチ、GPIO ポート 26 に抵抗と LED を繋ぎます。

CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";  
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));  
let port2;
```

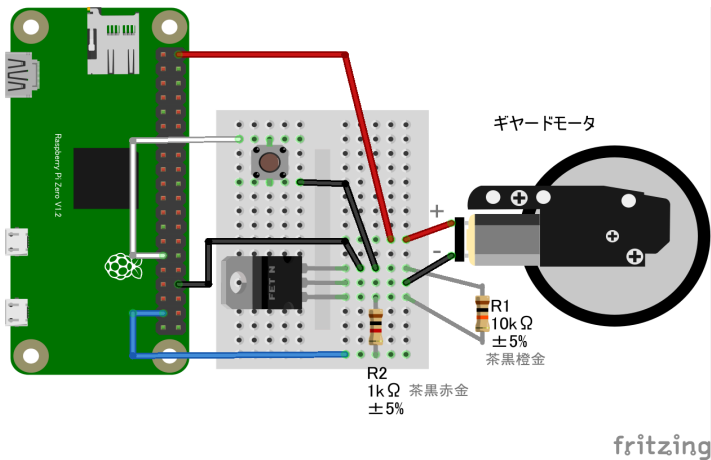
```
async function startGpio() {  
  const gpioAccess = await requestGPIOAccess();  
  
  port2 = gpioAccess.ports.get(26);  
  await port2.export("out");  
  
  const port = gpioAccess.ports.get(5);  
  await port.export("in");  
  port.onchange = showPort;  
}  
  
function showPort(ev){  
  console.log(ev.value);  
  if (ev.value==0){  
    port2.write(1);  
  } else {  
    port2.write(0);  
  }  
}  
  
startGpio();
```

GPIO スイッチとギヤードモーター

概要

- スイッチを押すとモーターが回転し、離すと停止
- 前項 LED 制御と同様の構成

配線図



GPIO ポート 5 にスイッチ、GPIO ポート 26 にモーター制御回路を繋ぎます。

CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
let port2;
```

```
async function startGpio() {  
  const gpioAccess = await requestGPIOAccess();  
  
  port2 = gpioAccess.ports.get(26);  
  await port2.export("out");  
  
  const port = gpioAccess.ports.get(5);  
  await port.export("in");  
  port.onchange = showPort;  
}  
  
function showPort(ev){  
  console.log(ev.value);  
  if (ev.value==0){  
    port2.write(1);  
  } else {  
    port2.write(0);  
  }  
}  
  
startGpio();
```

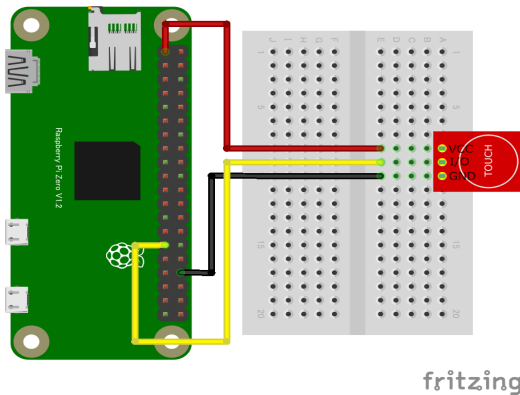
タッチセンサー

GPIOタッチセンサー

概要

- タッチや金属の近接で反応する静電容量式センサーの状態を検知
- 指でのタッチだけでなく金属の近接も感知(1~2mm の接近で検知)

配線図



CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

async function switchCheck() {
```



```
const gpioAccess = await requestGPIOAccess();
const port = gpioAccess.ports.get(5);

await port.export("in");
port.onchange = showPort;

}

function showPort(ev){
  console.log(ev.value);
}

switchCheck();
```

特記事項

- GPIO ポート 5 に接続、3.3 V 電源を使用
- 金属がセンサーのピンに触れてショートしないよう、テープで被覆して絶縁して下さい

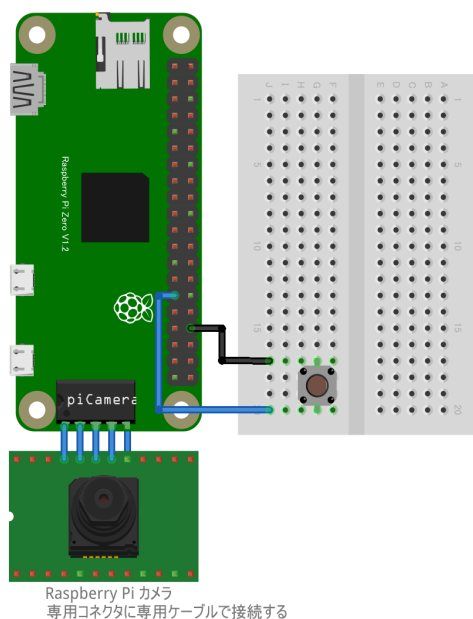
カメラ

GPIO スイッチによるカメラ撮影 (GPIO-Camera)

概要

- GPIO スイッチを押すとカメラで撮影し、画像ファイルを保存

配線図



fritzing

CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
// GPIO5のスイッチを押すと、Raspberry Pi Cameraで撮影し、ファイルに保存する

// ライブラリ pi-camera-connect をまずインストールする必要があります。readmeを参照してください。

import { StillCamera } from "pi-camera-connect";
import * as fs from "fs";

import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js";
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

const stillCamera = new StillCamera({
  width: 600,
  height: 600,
});

async function switchCheck() {
  const gpioAccess = await requestGPIOAccess();
  const port = gpioAccess.ports.get(5);

  await port.export("in");
  port.onchange = takeImage;
}

async function takeImage(ev) {
  if (ev.value == 1) {
    // 押し下げたときだけ撮影
    return;
  }
}
```

```

    const image = await stillCamera.takeImage();
    const fileName = "still-image-" + new Date().getTime() +
".jpg";
    console.log("sw:", ev.value, " takeImage:", fileName);
    fs.writeFileSync(fileName, image);
  }

  switchCheck();

```

特記事項

タクトスイッチは GPIO PORT5 に繋がります。

カメラは専用コネクタに専用ケーブルを使って接続し、更にセットアップが必要です。次章を参照してください

カメラのセットアップと動作確認

Raspberry Pi のカメラ^{*1}を API で直接操作する pi-camera-connect^{*2}を使った方法です。Pi-Camera^{*3}を使った方法(gist はこちら^{*4})より、大幅に高速に画像が取得できることを確認しています。

準備

- Raspberry Pi カメラモジュール
 - 例 1: KEYESTUDIO カメラモジュール^{*5}、例 2^{*6}、例 3^{*7}

1. <https://www.raspberrypi.com/documentation/accessories/camera.html>

2. <https://www.npmjs.com/package/pi-camera-connect>

3. <https://github.com/stetsmando/pi-camera>

4. <https://gist.github.com/satakagi/2c5be63d4759fd21eca939f507e7f7ef>

5. <https://www.amazon.co.jp/dp/B073RCXGQS/>

6. <https://www.amazon.co.jp/dp/B086MK17K5/>

7. <https://www.amazon.co.jp/dp/B08HVRB59N/>

- Zero 用ケーブル～上のモジュールは添付されているようです。
 - 無い場合は 別途調達^{*8}
- 接続のしかた^{*9} : Zero は専用ケーブルでつなぎます

Note

利用可能なカメラモジュールは v1、v3 です。Camera Module v2 には未対応です。また Raspberry Pi Zero 用 CHIRIMEN v1.4.0 未満をお使いの場合、Camera Module v3 には未対応です。Raspberry Pi Zero 用 CHIRIMEN v1.4.0 以上^{*10} をお使いください。

カメラの動作テスト

以下のコマンドで画像ファイルが保存されます:

```
raspistill -v --width 640 --height 480 -o test.jpg
```

Note

Raspberry Pi Zero 用 CHIRIMEN v1.4.0 以上^{*11} をお使いの場合、`rpicas-still --list-cameras` コマンドで利用可能なカメラの一覧を表示可能です:

```
$ rpicas-still --list-cameras
Available cameras
-----
```

8. <https://www.amazon.co.jp/gp/product/B07QH455KY/>

9. <https://projects.raspberrypi.org/ja-JP/projects/getting-started-with-picamera>

10. <https://github.com/chirimen-oh/chirimen-lite/releases>

11. <https://github.com/chirimen-oh/chirimen-lite/releases>

```
0 : ov5647 [2592x1944 10-bit GBGR] (/base/soc/i2c0mux/i2c@1/ov5647@36)
    Modes: 'SGBRG10_CSI2P' : 640x480 [58.92 fps - (16, 0)/2560x1920 crop]
                                1296x972 [43.25 fps - (0, 0)/2592x1944 crop]
                                1920x1080 [30.62 fps - (348, 434)/1928x1080 crop]
                                2592x1944 [15.63 fps - (0, 0)/2592x1944 crop]
```

詳細: Camera software - Raspberry Pi Documentation^{*12}

サンプル

pi-camera-connect のリポジトリ^{*13}の readme と同じ内容ですが、オプションを加えてみました

```
import { StillCamera } from "pi-camera-connect";
import * as fs from "fs";

// Take still image and save to disk
async function runApp() {
  const stillCamera = new StillCamera({
    width: 600,
    height: 600,
  });
  const image = await stillCamera.takeImage();
```

12. https://www.raspberrypi.com/documentation/computers/camera_software.html

13. <https://github.com/launchcodedev/pi-camera-connect>

```
fs.writeFileSync("still-image.jpg", image);
}

runApp();
```

dataURL を取得

```
import { StillCamera } from "pi-camera-connect";

// Take still image and get dataURI
async function runApp() {
  const stillCamera = new StillCamera({
    width: 200,
    height: 200,
  });
  var mime = "image/jpeg";
  var encoding = "base64";
  const image = await stillCamera.takeImage();
  const b64str = image.toString(encoding);
  const dataURL = "data:" + mime + ";" + encoding + "," + b64str;
  console.log(dataURL);
}

runApp();
```

Note

- dataURL^{*14} で画像を文字列化すれば比較的簡単にサーバに送信したりできるでしょう。
 - リモートカメラサンプル^{*15}
- 各種センサ (WebGPIO 経由で人感センサーなど)を使い、自動的に撮影、サーバにアップロードする仕組みなどもできるでしょう。

- pi-camera-connect のリポジトリ^{*16}
- この章はこちらの記事^{*17}を改変して作成されました。

14. https://developer.mozilla.org/ja/docs/Web/HTTP/Basics_of_HTTP/Data_URIs

15. https://tutorial.chirimen.org/pizero/esm-examples/#REMOTE_remote_camera

16. <https://github.com/launchcodedev/pi-camera-connect>

17. <https://gist.github.com/satakagi/1b5adc8dff8236421a593b93fa152222>

環境センサー

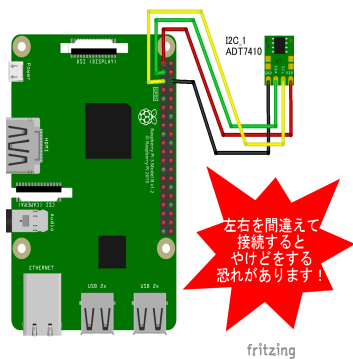
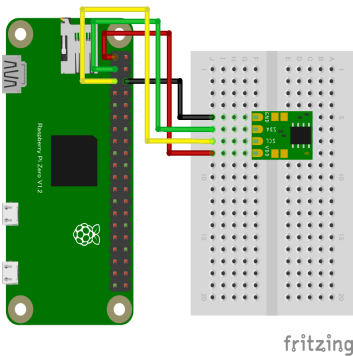
温度・湿度センサー

ADT7410 温度センサー

概要

- I²C 接続の温度センサーで、 $\pm 0.5^{\circ}\text{C}$ の高精度 ($0^{\circ}\text{C} \sim +70^{\circ}\text{C}$ の範囲) 測定可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/adt7410
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import ADT7410 from "@chirimen/adt7410";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

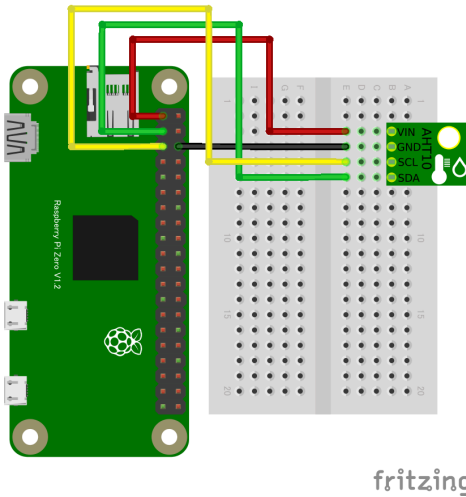
async function main() {
  const i2cAccess = await requestI2CAccess(); // i2cAccessを非同期で取得
  const port = i2cAccess.ports.get(1); // I2C I/Fの1番ポートを取得
  const adt7410 = new ADT7410(port, 0x48); // 取得したポートの0x48アドレスをADT7410ドライバで受信する
  await adt7410.init();
  for (;;) {
    // 無限ループ
    const value = await adt7410.read();
    console.log(`${value} degree`);
    await sleep(1000);
  }
}
```

AHT10 温湿度センサー

概要

- 温度と湿度を同時に計測可能なデジタルセンサー

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/aht10
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import AHT10 from "@chirimen/aht10";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const aht10 = new AHT10(port, 0x38);
  await aht10.init();

  while (true) {
    const { humidity, temperature } = await aht10.readData();
    ;
    console.log(
      [
        `Humidity: ${humidity.toFixed(2)}%`,
        `Temperature: ${temperature.toFixed(2)} degree`
      ].join(", ")
    );

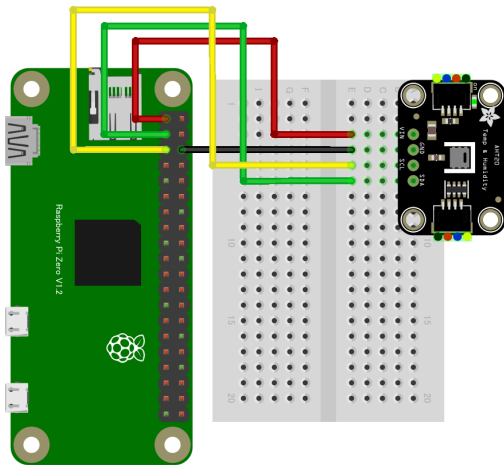
    await sleep(500);
  }
}
```

AHT20 温湿度センサー

概要

- 温度と湿度を同時に計測可能なデジタルセンサー

配線図



fritzing

CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/ahtx0
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "./node_modules/node-web-i2c/index.js";
import AHT20 from "@chirimen/ahtx0";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const aht20 = new AHT20(port, 0x38);
  await aht20.init();

  while (true) {
    const { humidity, temperature } = await aht20.readData()
    ;
    console.log(
      [
        `Humidity: ${humidity.toFixed(2)}%`,
        `Temperature: ${temperature.toFixed(2)} degree`
      ].join(", ")
    );

    await sleep(500);
  }
}
```

特記事項

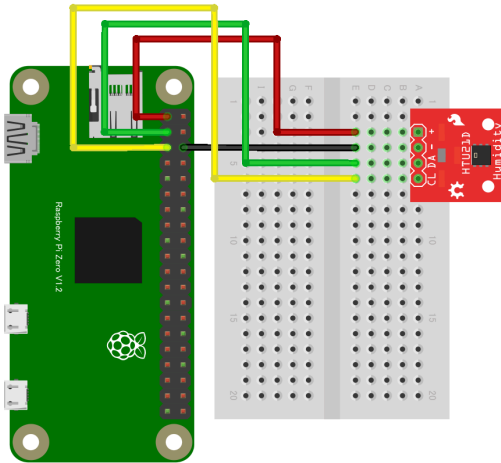
Note: このデバイスで使用するドライバahtx0.jsは、AHT20 及び AHT10 で使用可能

HTU21D 温湿度センサー

概要

- 高精度の温度・湿度センサーで、I²C 接続により簡単に利用可能
- コンパクトで応答速度も速く、さまざまな環境モニタリング用途に最適

配線図



fritzing

CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/htu21d
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import HTU21D from "@chirimen/htu21d";
const sleep = (msec) => new Promise((resolve) => setTimeout(
```

```
resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const htu21d = new HTU21D(port, 0x40);
  await htu21d.init();

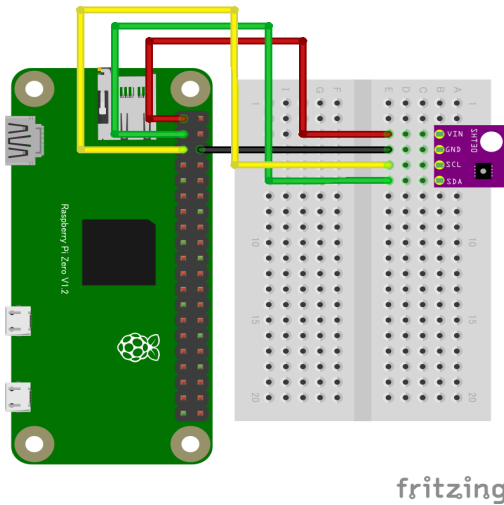
  while (true) {
    var temp = await htu21d.readTemperature();
    var humi = await htu21d.readHumidity();
    console.log("Temperature:", temp, " Humidity", humi)
    ;
    await sleep(1000);
  }
}
```


SHT30 温湿度センサー

概要

- I²C 接続のデジタル温湿度センサー
- 高精度かつ応答性が良く、空調や室内環境モニタリングに最適

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/sht30
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import SHT30 from "@chirimen/sht30";
const sleep = msec => new Promise(resolve => setTimeout(reso
```

```
lve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const sht30 = new SHT30(port, 0x44);
  await sht30.init();

  while (true) {
    const { humidity, temperature } = await sht30.readData();
    ;
    console.log(
      [
        `Humidity: ${humidity.toFixed(2)}%`,
        `Temperature: ${temperature.toFixed(2)} degree`
      ].join(", ")
    );

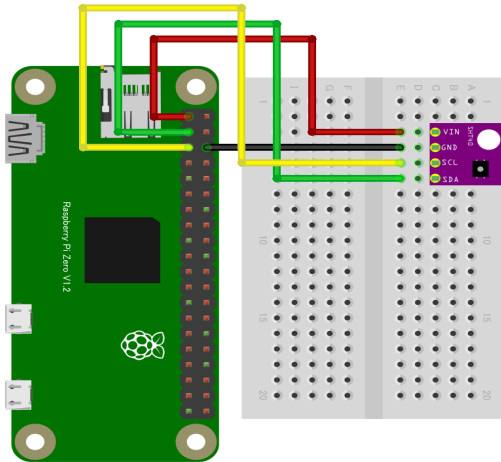
    await sleep(500);
  }
}
```

SHT40 温湿度センサー

概要

- I²C 接続のデジタル温湿度センサー
- 高精度かつ応答性が良く、空調や室内環境モニタリングに最適

配線図



fritzing

CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/sht40
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import SHT40 from "@chirimen/sht40";
const sleep = msec => new Promise(resolve => setTimeout(reso
```

```
lve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const sht40 = new SHT40(port, 0x44);
  await sht40.init();

  while (true) {
    const { humidity, temperature } = await sht40.readData();
    console.log(
      [
        `Humidity: ${humidity.toFixed(2)}%`,
        `Temperature: ${temperature.toFixed(2)} degree`
      ].join(", ")
    );

    await sleep(500);
  }
}
```

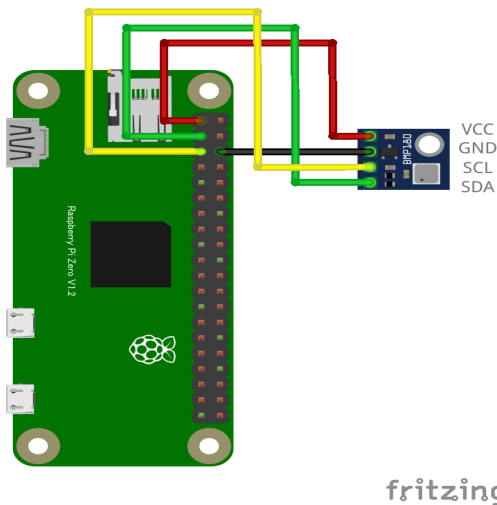
気圧センサー

BMP180 大気圧・温度センサー

概要

- 温度・気圧を 1 チップで計測可能な高性能センサー

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/bmp180
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";  
import BMP180 from "@chirimen/bmp180";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const readInterval = 1000;

  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const bmp180 = new BMP180(port, 0x77);
  await bmp180.init();
  for (;;) {
    const pressure = await bmp180.readPressure();
    const temperature = await bmp180.readTemperature();
    console.log(
      `Pressure: ${pressure.toFixed(2)} Pa, Temperature: ${temperature} degree.`
    );
    await sleep(readInterval);
  }
}
```

BMP280 温度・気圧センサー

概要

- 温度・気圧を 1 チップで計測可能な高性能センサー

配線図

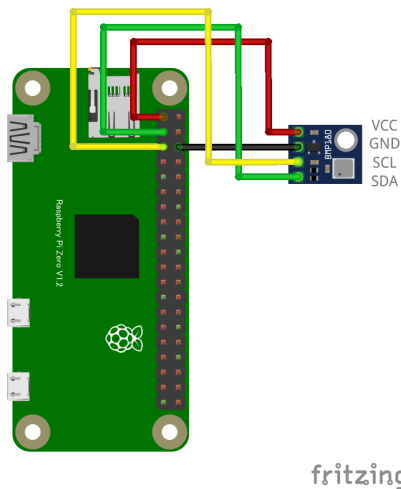


図 1: 配線図

CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/bmp280
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "./node_modules/node-web-i2c/index.js";  
import BMP280 from "@chirimen/bmp280";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const bmp280 = new BMP280(port, 0x76);
  await bmp280.init();

  while (true) {
    const data = await bmp280.readData();
    const pressure = data.pressure.toFixed(2);
    const temperature = data.temperature.toFixed(2);
    console.log(
      [`Temperature: ${temperature} degree`, `Pressure: ${pressure} hPa`].join(
        ", "
      )
    );

    await sleep(500);
  }
}
```


BME280 温度・湿度・気圧センサー

概要

- 温度・湿度・気圧を 1 チップで計測可能な高性能センサー

配線図

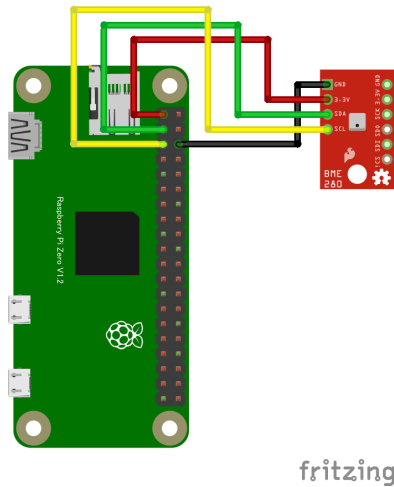


図 1: 配線図

CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/bme280
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";  
import BME280 from "@chirimen/bme280";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const bme280 = new BME280(port, 0x76);
  await bme280.init();

  while (true) {
    const data = await bme280.readData();
    const temperature = data.temperature.toFixed(2);
    const humidity = data.humidity.toFixed(2);
    const pressure = data.pressure.toFixed(2);
    console.log(
      [
        `Temperature: ${temperature} degree`,
        `Humidity: ${humidity} %`,
        `Pressure: ${pressure} hPa`
      ].join(", ")
    );
    await sleep(500);
  }
}
```

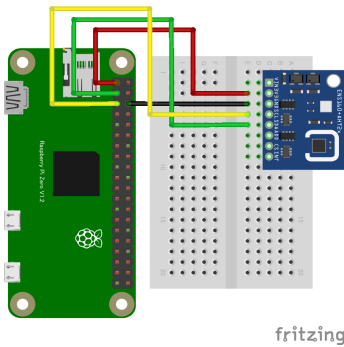
空気品質センサー

ENS160 空気質センサー

概要

- デジタルマルチガスセンサーで、室内空気質（IAQ）のモニタリングに特化
- 金属酸化物（MOX）技術を採用し、揮発性有機化合物（VOCs）や酸化性ガスの検出に優れた性能を発揮

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/ens160
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import ENS160 from "@chirimen/ens160";
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const ens160 = new ENS160(port, 0x53);
  await ens160.init();

  await ens160.set_temperature(22); // 温度を設定する(°C)
  await ens160.set_humidity(52); // 湿度を設定する(%)
  await sleep(1500);
  console.log(
    `setT:${(await ens160.get_temperature()).toFixed(2)}
[deg.] setH:${(await ens160.get_humidity()).toFixed(2)} [%]`
  );

  while (true) {
    const { AQI, TVOC, eCO2 } = await ens160.get_data();
    const mode = await ens160.get_mode();
    console.log(
      [
        `AQI(1-5):${AQI}`,
        `TVOC:${TVOC}ppb`,
        `eCO2:${eCO2}ppm`,
        `mode:${mode}(0:sleep,1:idle,2:standard)`,
      ].join(", ")
    );

    await sleep(500);
  }
}
```

特記事項

Note: 配線図のモジュールは、ENS160に加えて AHT20(温度湿度センサ)も載っている複合センサボードです。(ENS160 は温度と湿度の設定が必要) Amazonの商品例^{*1}

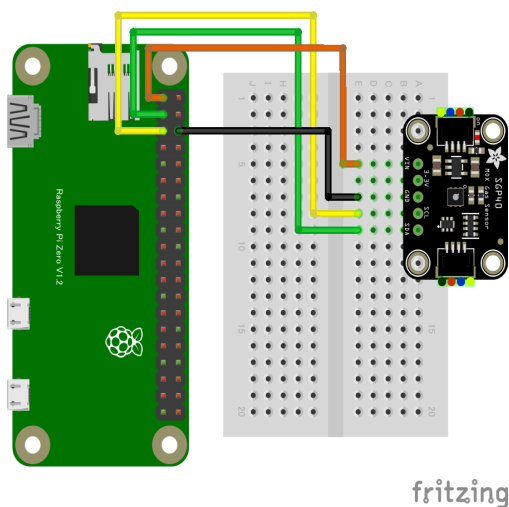
1. <https://www.amazon.co.jp/dp/B0D41R4V3Z>

SGP40 ガスセンサー

概要

- デジタル VOC（揮発性有機化合物）センサーで、室内空気質のモニタリングに特化
- 小型・低消費電力で、空気清浄機や換気システム、スマートホームデバイスなどへの組み込みに最適

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/sgp40
```

サンプルコード (main.js)

```

/* 各種ライブラリをインポート */
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js"; // WebGPIO
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js"; // WebI2C
import SGP40 from "@chirimen/sgp40"; // SGP40
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

main();

var sgp40;
async function main() {
  const i2cAccess = await requestI2CAccess();
  const ic2Port = i2cAccess.ports.get(1);
  sgp40 = new SGP40(ic2Port);
  await sgp40.init();
  while (true) {
    // var sgp = await sgp40.raw(); // 温度・湿度 内部で決め打ち・・(25℃,50%)
    var sgp = await sgp40.measureRaw(25, 50); // 温度・湿度決め打ち・・
    console.log("Gas:", sgp );
    await sleep(1000);
  }
}

```

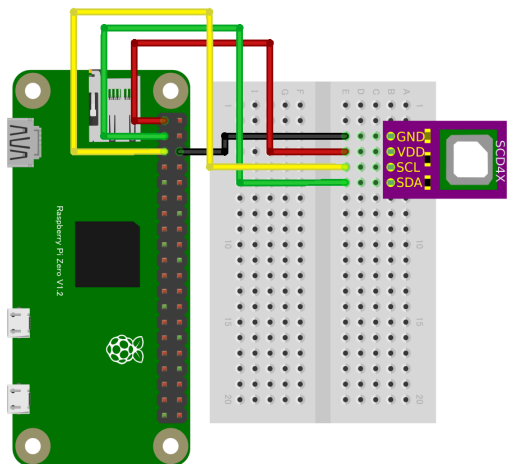
CO2 センサー

SCD40 CO2センサー

概要

- CO₂（二酸化炭素）濃度に加え、温度・湿度も取得可能なセンサー。
- NDIR 方式で高精度。

配線図



fritzing

CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/scd40
```


サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import SCD40 from "@chirimen/scd40";
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const scd40 = new SCD40(port, 0x62);
  await scd40.init();
  console.log(await scd40.serial_number());
  await scd40.start_periodic_measurement();

  // 値が出てくるまで数秒かかります
  // 測定が更新されると、updatedフラグがtrueになります
  while (true) {
    var data = await scd40.getData();
    console.log(data);
    await sleep(1000);
  }
}
```

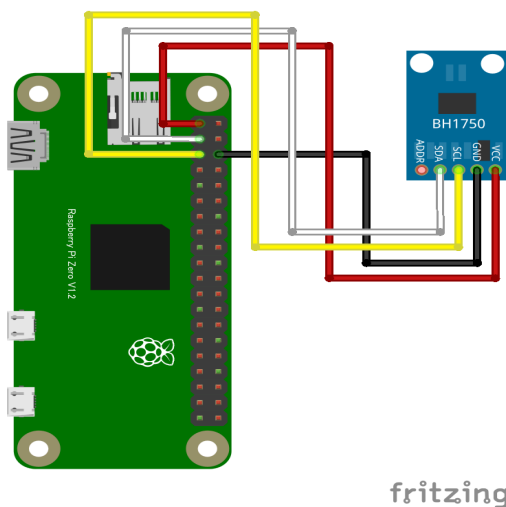
光センサー

BH1750 光センサー

概要

- 光の強さ（照度）をルクス単位で取得できるセンサー。

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/bh1750
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/  
index.js";  
import BH1750 from "@chirimen/bh1750";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const bh1750 = new BH1750(port, 0x23);
  await bh1750.init();

  while (true) {
    const lux = await bh1750.measure_high_res();
    console.log(lux.toFixed(3) + "lx");

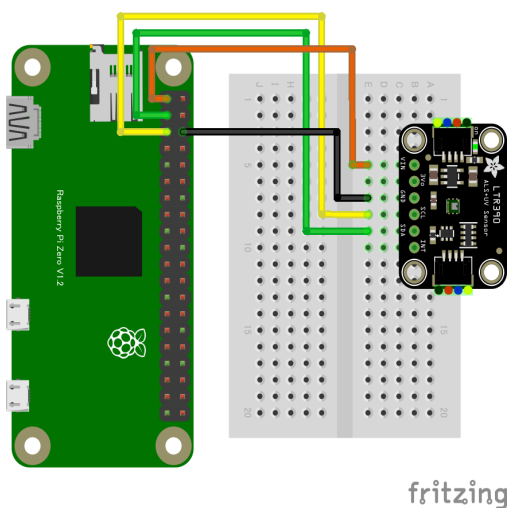
    await sleep(500);
  }
}
```

LTR390 UVセンサー

概要

- 高感度なデジタル紫外線（UV）および環境光センサー
- 小型で低消費電力、I²C 通信に対応しており、UV 強度や周囲の明るさを正確に測定可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/ltr390
```

サンプルコード (main.js)

```
/* 各種ライブラリをインポート */  
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js"; // WebGPIO
```

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js"; // WebI2C
import LTR390 from "@chirimen/ltr390"; // LTR390
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

main();

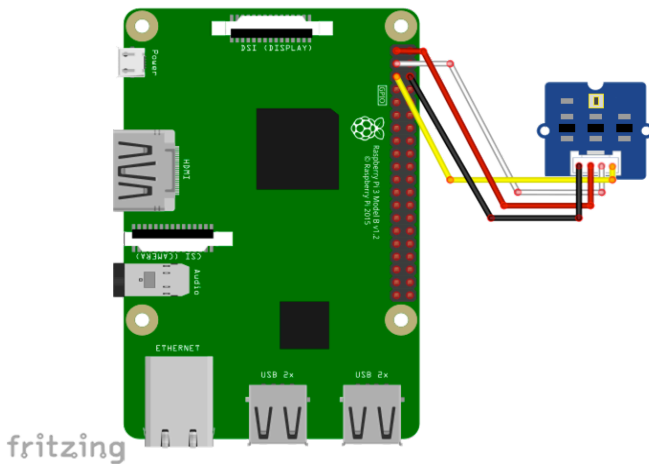
var ltr390;
async function main() {
  /* センサー初期化 */
  const i2cAccess = await requestI2CAccess();
  const ic2Port = i2cAccess.ports.get(1);
  ltr390 = new LTR390(ic2Port);
  await ltr390.init();
  while (true) {
    var UVS = await ltr390.UVS();
    console.log(UVS , " UVS");
    await sleep(1000);
  }
}
```

TSL2561 Grove Digital Light Sensor 光センサー

概要

- デジタル光センサーモジュール
- 屋内外の光環境のモニタリングや自動明るさ制御などに最適

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/grove-light
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/  
index.js";  
import GROVELIGHT from "@chirimen/grove-light";  
const sleep = msec => new Promise(resolve => setTimeout(reso
```

```
lve, msec));

main();

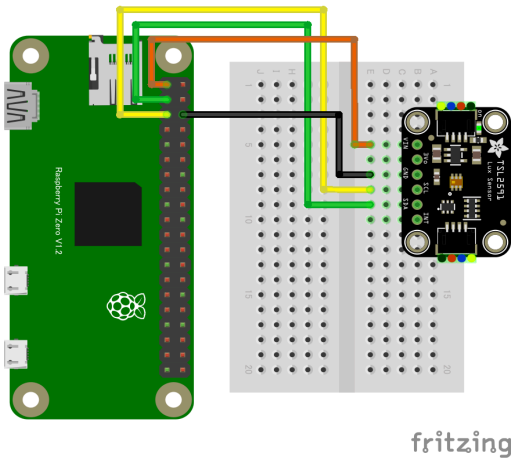
async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const grovelight = new GROVELIGHT(port, 0x29);
  await grovelight.init();
  for (;;) {
    try {
      const value = await grovelight.read();
      console.log(value);
    } catch (error) {
      console.error(" Error : ", error);
    }
    await sleep(200);
  }
}
```

TSL2591照度センサー

概要

- 高感度・広ダイナミックレンジのデジタル光センサー
- 光強度をデジタル信号に変換し、照度（lux）を高精度に測定可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/tsl2591
```

サンプルコード (main.js)

```
/* 各種ライブラリをインポート */  
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js"; // WebGPIO  
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js"; // WebI2C
```



```
import TSL2591 from "@chirimen/tsl2591"; // TSL2591
const sleep = (msec) => new Promise((resolve) => setTimeout(
  resolve, msec));

main();

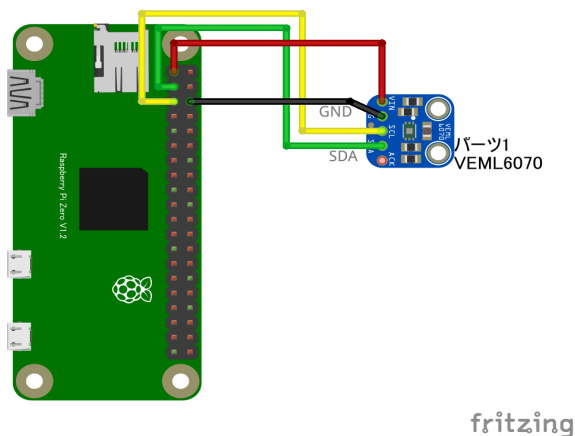
var tsl2591;
async function main() {
  /* センサー初期化 */
  const i2cAccess = await requestI2CAccess();
  const ic2Port = i2cAccess.ports.get(1);
  tsl2591 = new TSL2591(ic2Port);
  await tsl2591.init();
  while (true) {
    var lux = await tsl2591.Lux();
    console.log(lux , " Lux");
    await sleep(1000);
  }
}
```

VEML6070 UV センサー

概要

- 高感度な紫外線（UV）センサー
- 特に UVA 領域（波長 320～410nm、ピーク感度 355nm）の検出に特化

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/veml6070
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import VEML6070 from "@chirimen/veml6070";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const veml6070 = new VEML6070(port);
  await veml6070.init();
  for (;;) {
    const value = await veml6070.read();
    console.log(value);
    await sleep(200);
  }
}
```

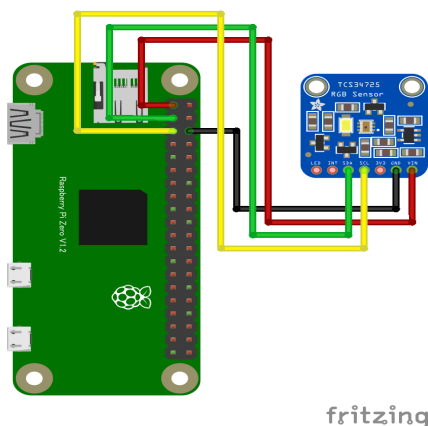
色センサー

TCS34725 RGB カラーセンサー

概要

- 赤 (Red)、緑 (Green)、青 (Blue)、および透明光 (Clear) の4つのチャンネルを装備
- 赤外線 (IR) ブロッキングフィルタを内蔵しており、正確な色測定が可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/tcs34725
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";  
import TCS34725 from "@chirimen/tcs34725";
```

```

const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const tcs34725 = new TCS34725(port, 0x29);
  await tcs34725.init();

  // You can select the value of gain from 1, 4, 16 or 60.
  await tcs34725.gain(4);

  while (true) {
    const data = await tcs34725.read();
    console.log(
      [
        `R: ${data.r}`,
        `G: ${data.g}`,
        `B: ${data.b}`,
        `Clear Light: ${data.c}`
      ].join(", ")
    );

    await sleep(500);
  }
}

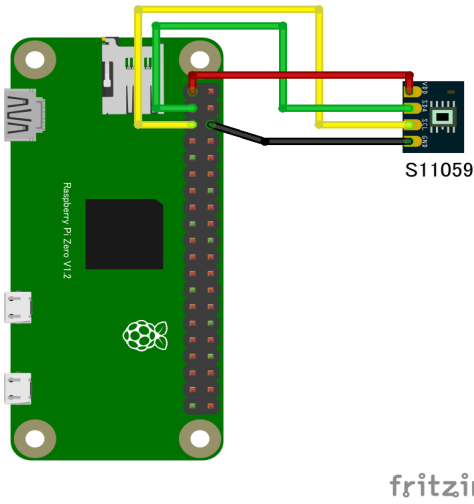
```

S11059 デジタルカラーセンサー

概要

- 高精度なデジタルカラーセンサー
- 赤 (Red)、緑 (Green)、青 (Blue)、および赤外線 (IR) の4つの波長帯域を個別に測定可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/s11059
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";  
import S11059 from "@chirimen/s11059";
```

```

const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const s11059 = new S11059(port, 0x2a);
  await s11059.init();
  for (;;) {
    try {
      const values = await s11059.readR8G8B8();
      const red = values[0] & 0xff;
      const green = values[1] & 0xff;
      const blue = values[2] & 0xff;
      const gain_level = values[3];
      console.log(`R:${red} G:${green} B:${blue} GAIN: ${gain_level}`);
    } catch (error) {
      console.error("READ ERROR:" + error);
    }
    await sleep(1000);
  }
}

```

モーション・位置センサー

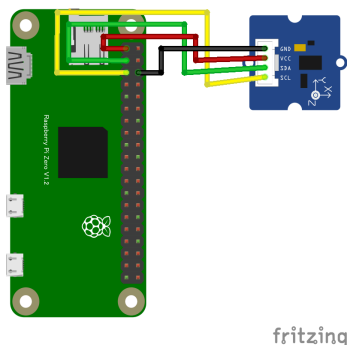
加速度・ジャイロセンサー

ADXL345 3 軸加速度センサー

概要

- 3 軸加速度センサー ADXL345 搭載 Grove システム対応モジュール
- X、Y、Z の 3 軸方向の加速度を高精度に測定可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/grove-accelerometer
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";  
import GROVEACCELEROMETER from "@chirimen/grove-acceleromete
```



```

r";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const groveaccelerometer = new GROVEACCELEROMETER(port, 0x53);
  await groveaccelerometer.init();
  for (;;) {
    try {
      const values = await groveaccelerometer.read();
      console.log(`ax: ${values.x}, ay: ${values.y}, az: ${values.z}`);
    } catch (err) {
      console.error("READ ERROR:" + err);
    }
    await sleep(1000);
  }
}

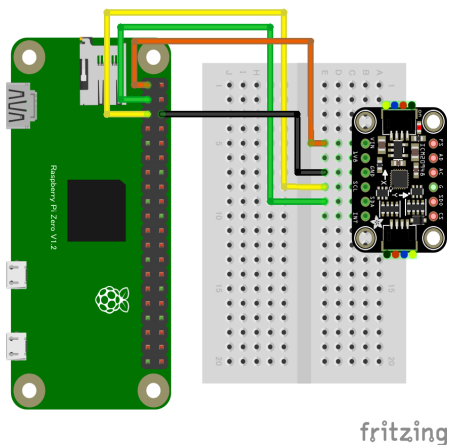
```

ICM20948 9軸センサー

概要

- 超低消費電力 9 軸モーショントラッキングデバイスで、スマートフォン、ウェアラブルデバイス、IoT 機器などに最適
- 3 軸加速度計、3 軸ジャイロスコプ、3 軸磁力計 (AK09916) を統合し、さらにオンボードのデジタルモーションプロセッサ (DMP™) を装備

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/icm20948
```

サンプルコード (main.js)

```
/* 各種ライブラリをインポート */  
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js"; // WebGPIO  
import { requestI2CAccess } from "../node_modules/node-web-i2c"
```

```

c/index.js"; // WebI2C
import ICM20948 from "@chirimen/icm20948"; // ICM20948
const sleep = (msec) => new Promise((resolve) => setTimeout(
  resolve, msec));

main();

var icm20948;
async function main() {
  /* センサー初期化 */
  const i2cAccess = await requestI2CAccess();
  const ic2Port = i2cAccess.ports.get(1);
  icm20948 = new ICM20948(ic2Port);
  await icm20948.init();
  while (true) {
    var icm = await icm20948.getdata();
    console.log("/-----
-----/");
    console.log(`Roll = ${icm[0].toFixed(2)} , Pitch = $
${icm[1].toFixed(2)} , Yaw = ${icm[2].toFixed(2)}`);
    console.log(`Acceleration: X = ${icm[3]}, Y = ${icm[
4]}, Z = ${icm[5]}`);
    console.log(`Gyroscope:      X = ${icm[6]} , Y = ${ic
m[7]} , Z = ${icm[8]}`);
    console.log(`Magnetic:      X = ${icm[9]} , Y = ${ic
m[10]} , Z = ${icm[11]}`);
    await sleep(1000);
  }
}

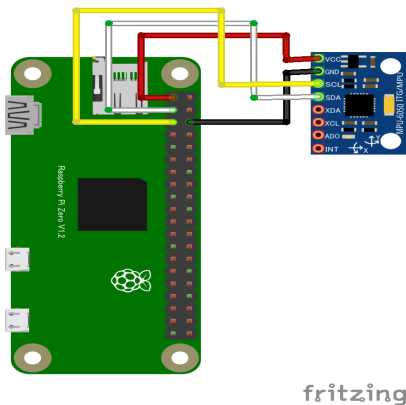
```

MPU6050 3 軸ジャイロ 3 軸加速度 複合センサー

概要

- 6 軸モーションセンサーで、3 軸加速度計と 3 軸ジャイロスコプを 1 チップに統合
- デバイスの動きや傾きを高精度に検出可能、ドローン、ロボット、ウェアラブルデバイスなど、さまざまなアプリケーションで広く利用

回路図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/mpu6050
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import MPU6050 from "@chirimen/mpu6050";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  var i2cAccess = await requestI2CAccess();
  var port = i2cAccess.ports.get(1);
  var mpu6050 = new MPU6050(port, 0x68);
  await mpu6050.init();
  while (true) {
    const data = await mpu6050.readAll();
    const temperature = data.temperature.toFixed(2);
    const g = [data.gx, data.gy, data.gz];
    const r = [data.rx, data.ry, data.rz];
    console.log(
      [
        `Temperature: ${temperature} degree`,
        `Gx: ${g[0]}, Gy: ${g[1]}, Gz: ${g[2]}`,
        `Rx: ${r[0]}, Ry: ${r[1]}, Rz: ${r[2]}`
      ].join("\n")
    );

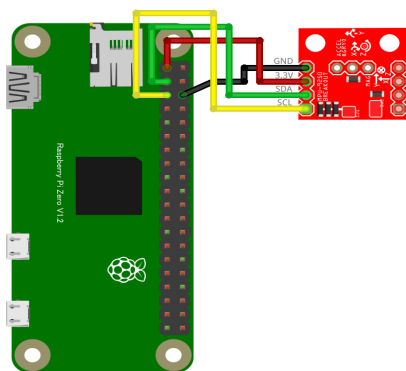
    await sleep(500);
  }
}
```

MPU9250 3 軸ジャイロ 3 軸加速度 3 軸 磁気複合センサー

概要

- 9 軸モーションセンサで、3 軸加速度計、3 軸ジャイロ스코プ、および 3 軸磁力計を 1 チップに統合したモジュール
- デバイスの動きや傾きを高精度に検出可能、ドローン、ロボット、ウェアラブルデバイスなど、さまざまなアプリケーションで広く利用

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/ak8963
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import MPU6500 from "@chirimen/mpu6500";
import AK8963 from "@chirimen/ak8963";
```

```

const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const mpu6500 = new MPU6500(port, 0x68);
  const ak8963 = new AK8963(port, 0x0c);
  await mpu6500.init();
  await ak8963.init();

  while (true) {
    const g = await mpu6500.getGyro();
    const r = await mpu6500.getAcceleration();
    const h = await ak8963.readData();
    console.log(
      [
        `Gx: ${g.x}, Gy: ${g.y}, Gz: ${g.z}`,
        `Rx: ${r.x}, Ry: ${r.y}, Rz: ${r.z}`,
        `Hx: ${h.x}, Hy: ${h.y}, Hz: ${h.z}`
      ].join("\n")
    );

    await sleep(500);
  }
}

```

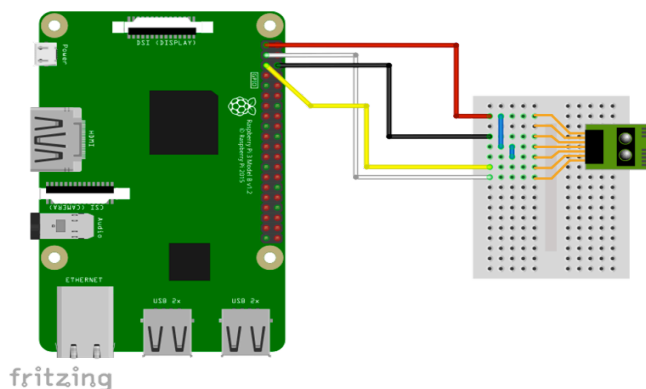
距離センサー

GP2Y0E03 測距センサー 40 mm - 0.1 m

概要

- 小型・高精度な赤外線測距センサーモジュールで、アナログおよびI²C出力に対応
- 三角測量の原理を利用し、赤外線 LED と CMOS イメージセンサを組み合わせ、対象物までの距離を 4cm から 50cm の範囲で測定可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/gp2y0e03
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "node-web-i2c";
import GP2Y0E03 from "@chirimen/gp2y0e03";
const sleep = msec => new Promise(resolve => setTimeout(reso
```



```
lve, msec));

main();

async function main() {
  try {
    const i2cAccess = await requestI2CAccess();
    const port = i2cAccess.ports.get(1);
    const sensor_unit = new GP2Y0E03(port, 0x40);
    await sensor_unit.init();

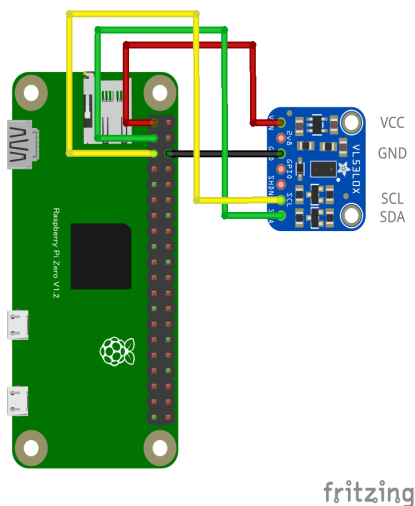
    while (1) {
      try {
        const distance = await sensor_unit.read();
        if (distance != null) {
          console.log("Distance:" + distance + "cm");
        } else {
          console.log("out of range");
        }
      } catch (err) {
        console.error("READ ERROR:" + err);
      }
      await sleep(500);
    }
  } catch (err) {
    console.error("GP2Y0E03 init error");
  }
}
```

VL53L0X レーザー測距センサー 30 mm - 2 m

概要

- ToF（Time of Flight）方式で 30 mm - 2 m 程度までの距離を非接触で測定可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/vl53l0x
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "./node_modules/node-web-i2c/index.js";
import VL53L0X from "@chirimen/vl53l0x";
const sleep = msec => new Promise(resolve => setTimeout(reso
```

```
lve, msec));

main();

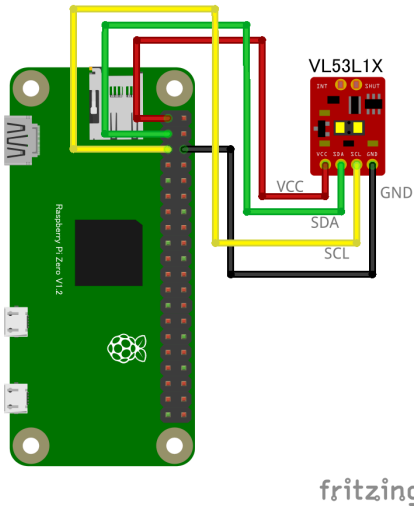
async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const vl = new VL53L0X(port, 0x29);
  await vl.init(); // for Long Range Mode (<2m) : await vl.i
nit(true);
  for (;;) {
    const distance = await vl.getRange();
    console.log(`${distance} [mm]`);
    await sleep(500);
  }
}
```

VL53L1X レーザー距離センサー

概要

- ToF（Time of Flight）方式で、距離を非接触で測定可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/vl53l1x
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import VL53L1X from "@chirimen/vl53l1x";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const vl53l1x = new VL53L1X(port, 0x29);

  // Mode: short, medium, long
  await vl53l1x.init("short");

  // Necessary to start measurement
  await vl53l1x.startContinuous();

  while (true) {
    const distance = await vl53l1x.read();
    console.log(distance.toFixed(2) + " mm");
    await sleep(500);
  }
}
```

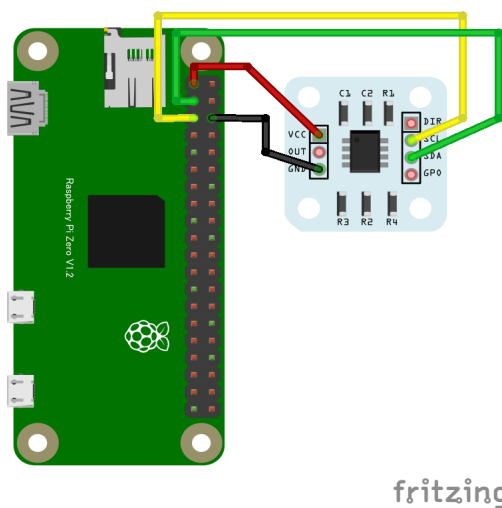
角度センサー

AS5600 磁気式角度センサー

概要

- ブレークアウトボードを購入すると付属している円盤状の磁石をチップに近接配置。
- この磁石の回転角を非接触で検出できるセンサー

配線図



CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
// AS5600(磁気式角度センサ)を使う
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import AS5600 from "../as5600.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

var as5600;

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  as5600 = new AS5600(port);
  await as5600.init();
  var stat = await as5600.getStatus();
  console.log("stat:" , stat);
  startMeasurement();
}

async function startMeasurement() {
  for (; ;) {
    var rawAngle = await as5600.getAngle();
    console.log("rawA:",rawAngle);
    await sleep(1000);
  }
}
```

特記事項

- 円盤状磁石の中心を回転軸とし、この角度を検出して、チップの中心と円盤状磁石の中心を一致させる
- チップ表面と磁石の距離は 0.5～3mm 程度にする (参照情報(データシート 33 ページ))*¹)
- DIR ピンを VCC に接続すると角度の方向が逆向きになる
- ブレークアウトボード：※1*², ※2*³
- ソフトウェア
 - `as5600.js` *⁴, main.js 両方とも myApp ディレクトリ直下に置く
- AS5600 データシート*⁵

1. https://ams.com/documents/20143/36005/AS5600_DS000365_5-00.pdf#page=34

2. <https://electronicwork.shop/items/64205dc6cd92fe0096fb7d5c>

3. <https://www.switch-science.com/products/3493>

4. <https://raw.githubusercontent.com/chirimen-oh/chirimen-drivers/master/packages/as5600/as5600.js>

5. https://ams.com/documents/20143/36005/AS5600_DS000365_5-00.pdf

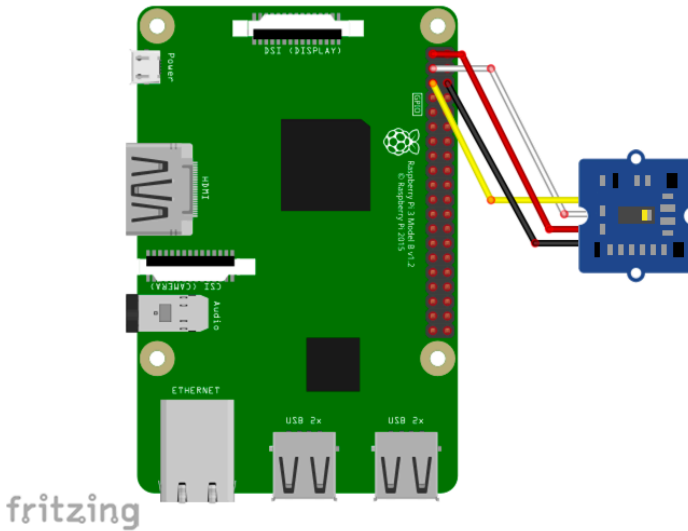
ジェスチャーセンサー

PAJ7620 Grove Gesture ジェスチャー認識センサー

概要

- 近接、ジェスチャーなど多機能なセンサー。

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/grove-gesture
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "./node_modules/node-web-i2c/
index.js";
import PAJ7620 from "@chirimen/grove-gesture";
const sleep = msec => new Promise(resolve => setTimeout(reso
lve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const gesture = new PAJ7620(port, 0x73);
  await gesture.init();

  for (;;) {
    const v = await gesture.read();
    console.log(v);
    await sleep(1000);
  }
}
```

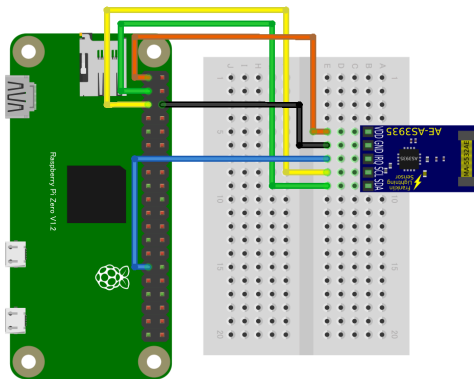
雷センサー

AS3935 雷センサー

概要

- IC に内蔵された雷検出アルゴリズムが入力信号をチェックし暴風雨の頭部までの距離を推定
- サンプルでは、I2C バスに加えて、GPIO ポート 5 を雷検出のトリガーとして使用

配線図



fritzing

CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/as3935
```

サンプルコード (main.js)

```
/* 各種ライブラリをインポート */
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js"; // WebGPIO
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js"; // WebI2C
import AS3935 from "@chirimen/as3935"; // AS3935
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

var as3935;
async function main() {
  /* 雷センサー初期化 */
  const i2cAccess = await requestI2CAccess();
  const ic2Port = i2cAccess.ports.get(1);
  as3935 = new AS3935(ic2Port);
  await as3935.init();
  await as3935.reset();
  await sleep(100);
  await as3935.set_indoors(true);
  await as3935.set_noise_floor(0);
  await as3935.calibrate(0x0f);

  /* 雷センサーインタラプト用GPIO初期化 */
  const gpioAccess = await requestGPIOAccess();
  const as3935_interruptPort = gpioAccess.ports.get(5);
  await as3935_interruptPort.export("in");
  as3935_interruptPort.onchange = as3935_sens;
}
```

```

async function as3935_sens(ev) {
  if (ev.value == 0) { return }
  console.log("interruptPort:", ev.value);
  await sleep(10);
  var reason = await as3935.get_interrupt();
  if (reason == 0x01) {
    console.log("Noise level too high - adjusting");
    await as3935.raise_noise_floor();
  } else if (reason == 0x04) {
    console.log("Disturber detected - masking");
    await as3935.set_mask_disturber(true);
  } else if (reason == 0x08) {
    var now = new Date().toLocaleString();
    var distance = await as3935.get_distance();
    console.log(`Detect lightning! It was ${distance} Km
away. (${now}) `);
  }
}

```

特記事項

- しっかりと動作確認ができていません。(雷検出の機会が少ないため) 動作確認にご協力ください。
- ISSUEにレポートを記載いただけると幸いです

使用するブレイクアウトボード

- I2C スレーブアドレス、**0x03** が設定されているものを使用しています。
- 秋月電子通商 AE-AS3935 がこれに相当します。: 製品ページ^{*1}

1. <https://akizukidenshi.com/catalog/g/gK-08685/>

I2Cデバイスの検出

- I2Cアドレス `0x03` のデバイスは、通常の `i2cdetect` コマンドでは検出できません（標準範囲外アドレス） `-a` オプションが必要です。コマンドプロンプトから、以下のコマンドで検出してください。
- `i2cdetect -y -a 1`

ディスプレイ・LED

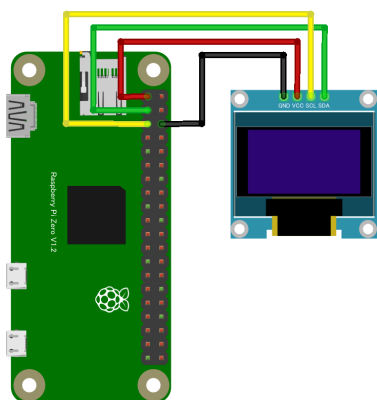
OLED ディスプレイ

SSD1306 OLED ディスプレイ

概要

- I²C 接続の小型モノクロ OLED ディスプレイ

配線図



fritzing

CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/grove-oled-display
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "./node_modules/node-web-i2c/
index.js";
import OledDisplay from "@chirimen/grove-oled-display";
const sleep = msec => new Promise(resolve => setTimeout(reso
lve, msec));

main();

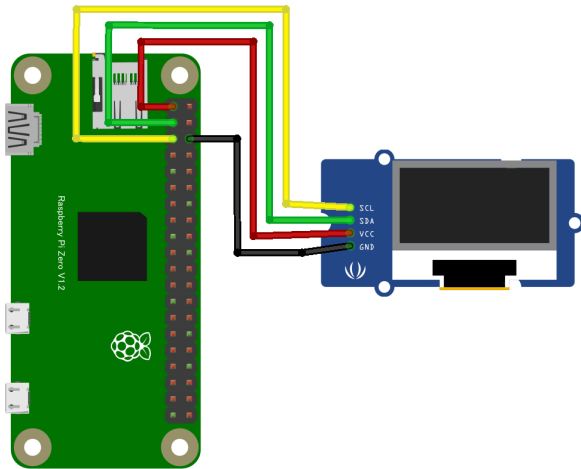
async function main() {
  const i2cAccess = await requestI2CAccess();
  console.log("initializing...");
  const port = i2cAccess.ports.get(1);
  const display = new OledDisplay(port);
  await display.init(true); // SSD1306(grove-oled-display)の
場合はtrueを設定(SSD1308の場合はパラメータ不要)
  display.clearDisplayQ();
  await display.playSequence();
  console.log("drawing text...");
  display.drawStringQ(0, 0, "Hello");
  display.drawStringQ(1, 0, "Real");
  display.drawStringQ(2, 0, "World");
  await display.playSequence();
  console.log("completed");
}
```


SSD1308 Grove OLED ディスプレイ

概要

- I²C 接続の Grove OLED ディスプレイ

配線図



fritzing

CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/grove-oled-display
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import OledDisplay from "@chirimen/grove-oled-display";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  console.log("initializing...");
  const port = i2cAccess.ports.get(1);
  const display = new OledDisplay(port);
  await display.init(); // SSD1308の場合はパラメータ不要(SSD130
6(grove-oled-display)の場合はtrueを設定)
  display.clearDisplayQ();
  await display.playSequence();
  console.log("drawing text...");
  display.drawStringQ(0, 0, "Hello");
  display.drawStringQ(1, 0, "Real");
  display.drawStringQ(2, 0, "World");
  await display.playSequence();
  console.log("completed");
}
```

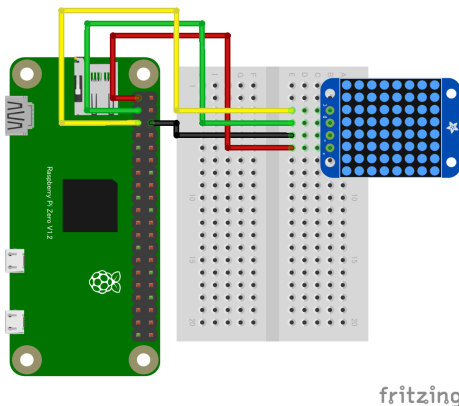
LED ドライバー

HT16K33 LEDマトリクスドライバ

概要

- HT16K33 LED マトリクスドライバ、8x8 LED マトリクスモジュール一体型や 7 セグメント LED 一体型等にも対応

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/ht16k33
```

サンプルコード (main.js)

```
// Adafruit Mini 8x8 LED Matrix w/I2C Backpack  
// https://www.switch-science.com/products/1493 , https://www.adafruit.com/product/870  
// or
```

```
// Ks0064 keyestudio I2C 8x8 LED Matrix HT16K33
// https://ja.aliexpress.com/item/32886174149.html

import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import HT16K33 from "@chirimen/ht16k33";
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

main();

const iconPattern =[
0,0,1,1,1,1,0,0,
0,1,0,0,0,0,1,0,
1,0,1,0,0,1,0,1,
1,0,0,0,0,0,0,1,
1,0,1,0,0,1,0,1,
1,0,0,1,1,0,0,1,
0,1,0,0,0,0,1,0,
0,0,1,1,1,1,0,0,
]; // スマイルマーク

const iconPattern2 =[
1,0,0,0,0,0,0,1,
0,1,0,0,0,0,1,0,
0,0,1,1,1,1,0,0,
0,1,1,1,1,1,1,0,
1,1,0,1,1,0,1,1,
0,1,1,1,1,1,1,0,
0,0,1,0,0,1,0,0,
1,1,0,0,0,0,1,1
]; // インベーター

const iconPattern3 =[
0,1,0,0,0,0,1,0,
```

```

1,1,1,1,1,1,1,0,
1,0,0,1,0,0,1,0,
1,1,0,1,1,0,1,0,
1,0,0,1,0,0,1,0,
1,1,1,1,1,1,1,0,
0,1,1,1,1,1,1,1,
0,1,0,1,0,1,0,1
]; // 犬

async function main() {
    const i2cAccess = await requestI2CAccess();
    const port = i2cAccess.ports.get(1);
    const ht = new HT16K33(port);
    await ht.init();

    //await ht.set_blink(ht.HT16K33_BLINK_1HZ);
    //await ht.set_brightness(6);

    while(true){
        ht.set_8x8_array(iconPattern);
        await ht.write_display();
        await sleep(1000);

        ht.set_8x8_array(iconPattern2);
        await ht.write_display();
        await sleep(1000);

        ht.set_8x8_array(iconPattern3);
        await ht.write_display();
        await sleep(1000);

        /** LEDを一個ずつ設定する関数の使用例
        for ( var i = 0 ; i < 128 ; i++ ){
            ht.set_led(i, 1);
        }

```

```
    await ht.write_display();  
    await sleep(1000);  
    **/  
  }  
}
```

特記事項

- Note: 3.3V電源でも、あまり明るくはなりませんが動作するようです。

8x8 マトリクスLED モジュール(デフォルト設定)

いずれでも動作(ピン配置は異なる)

- Adafruit Mini 8x8 LED Matrix w/I2C Backpack
 - <https://www.switch-science.com/products/1493>^{*1}
 - <https://www.adafruit.com/product/870>^{*2}
- keystudio Ks0064 I2C 8x8 LED Matrix HT16K33
 - <https://ja.aliexpress.com/item/32886174149.html>^{*3}
- サンプルコード [./main.js](#)

Aitendo 製 8x8 マトリクスLED モジュール

Adafruit、keystudio とマトリクスの結線が異なりますが、設定変更により使用可能です。

-
1. <https://www.switch-science.com/products/1493>
 2. <https://www.adafruit.com/product/870>
 3. <https://ja.aliexpress.com/item/32886174149.html>

- 対応品リスト

- <https://www.aitendo.com/product/12822>^{*4}
- <https://www.aitendo.com/product/12850>^{*5}
- <https://www.aitendo.com/product/12823>^{*6}

- サンプルコード [./main-ht16k33_8x8aitendo.js](#)

- aitendo 製モジュール用の設定変更関数を呼び出して使います。

4桁7セグメントLEDモジュール

配線が同じであれば、amazon や aliexpress 等で販売されているジェネリック品でも使用できると思います。(動作は Aitendo 製品で確認)

- 対応品：aitendo

- <https://www.aitendo.com/product/14540>^{*7}
- <https://www.aitendo.com/product/18690>^{*8}
- <https://www.aitendo.com/product/20785>^{*9}

- サンプルコード [./main-ht16k33_7seg.js](#)

4. <https://www.aitendo.com/product/12822>

5. <https://www.aitendo.com/product/12850>

6. <https://www.aitendo.com/product/12823>

7. <https://www.aitendo.com/product/14540>

8. <https://www.aitendo.com/product/18690>

9. <https://www.aitendo.com/product/20785>

4桁 14セグメントLEDモジュール

数字やアルファベットが表示可能なLEDモジュール。配線が同じであれば、amazonやaliexpress等で販売されているジェネリック品でも使用できると思います。（動作はAitendo製品で確認）

- 対応品：aitendo
 - <https://www.aitendo.com/product/20812>^{*10}
 - <https://www.aitendo.com/product/20811>^{*11}
- サンプルコード [./main-ht16k33_14seg.js](#)

その他

- set_led関数を使用することで、任意のLEDマトリクスに対応できます

10. <https://www.aitendo.com/product/20812>

11. <https://www.aitendo.com/product/20811>

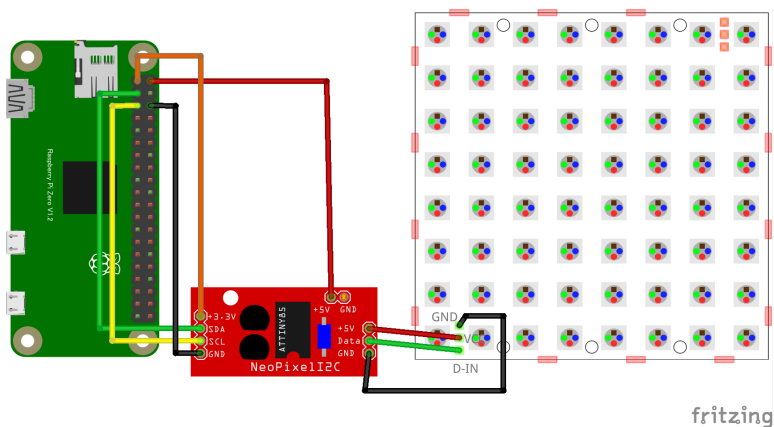
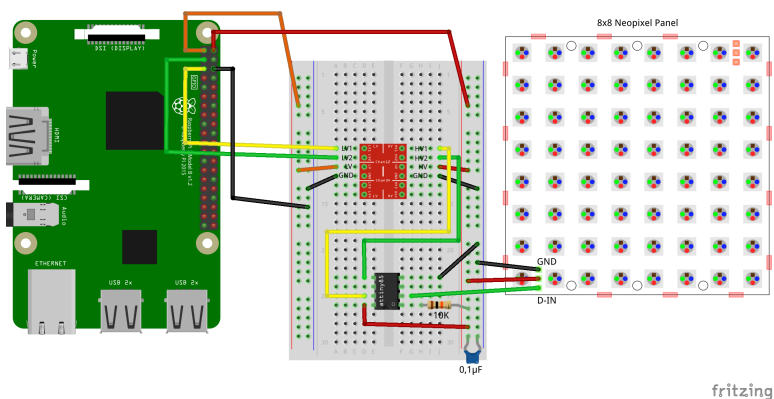
NeoPixel

Neopixel LED

概要

- 個別に制御可能な RGB（赤・緑・青）LED のブランド名
- 主に WS2812B や SK6812 といった制御 IC を内蔵した LED を使用
- 1 本のデータ線で複数の LED を直列に接続し、個別に制御できるのが特徴

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/neopixel-i2c
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import NPIX from "@chirimen/neopixel-i2c";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

const neoPixels = 7; // LED個数

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const npix = new NPIX(port, 0x41);
  await npix.init(neoPixels);

  await nPixTest1(npix);

  await nPixTest2(npix);

  await nPixTest3(npix, pattern4);

  await npix.setGlobal(0, 0, 0);
}

async function nPixTest1(npix) {
  // 全LEDを同じ色にするケース
```

```

    await npix.setGlobal(10, 0, 0);
    await sleep(200);
    await npix.setGlobal(0, 10, 0);
    await sleep(200);
    await npix.setGlobal(0, 0, 10);
    await sleep(200);
    await npix.setGlobal(0, 20, 20);
    await sleep(200);
    await npix.setGlobal(20, 0, 20);
    await sleep(200);
    await npix.setGlobal(20, 20, 0);
    await sleep(200);
    await npix.setGlobal(20, 20, 20);
    await sleep(200);
    await npix.setGlobal(0, 0, 0);
  }

  // パターンはRRGGBB の並びで
  const pattern0 = [0xff0000, 0x00ff00, 0x0000ff, 0xff0000, 0x
00ff00, 0x0000ff, 0xff0000];
  const pattern1 = [0x000000, 0x00ff00, 0x0000ff, 0xff0000, 0x
00ff00, 0x0000ff, 0xff0000];
  const pattern2 = [0x000000, 0x000000, 0x0000ff, 0xff0000, 0x
00ff00, 0x0000ff, 0xff0000];
  const pattern3 = [0x000000, 0x000000, 0x000000, 0xff0000, 0x
00ff00, 0x0000ff, 0xff0000];
  const pattern4 = [0x000000, 0x000000, 0x000000, 0x000000, 0x
00ff00, 0x0000ff, 0xff0000];
  const pattern5 = [0x000000, 0x000000, 0x000000, 0x000000, 0x
000000, 0x0000ff, 0xff0000];
  const pattern6 = [0x000000, 0x000000, 0x000000, 0x000000, 0x
000000, 0x000000, 0xff0000];
  const pattern7 = [0x000000, 0x000000, 0x000000, 0x000000, 0x
000000, 0x000000, 0x000000];

```

```
async function setPattern(npix, pattern) {  
  // パターン設定  
  const grbArray = [];  
  for (const color of pattern) {  
    const r = color >> 16 & 0xff;  
    const g = color >> 8 & 0xff;  
    const b = color & 0xff;  
    grbArray.push(g);  
    grbArray.push(r);  
    grbArray.push(b);  
  }  
  await npix.setPixels(grbArray);  
}
```

```
async function nPixTest2(npix) {  
  // パターンを変化させる  
  await setPattern(npix, pattern0);  
  await sleep(200);  
  await setPattern(npix, pattern1);  
  await sleep(200);  
  await setPattern(npix, pattern2);  
  await sleep(200);  
  await setPattern(npix, pattern3);  
  await sleep(200);  
  await setPattern(npix, pattern4);  
  await sleep(200);  
  await setPattern(npix, pattern5);  
  await sleep(200);  
  await setPattern(npix, pattern6);  
  await sleep(200);  
  await setPattern(npix, pattern7);  
}
```

```
async function nPixTest3(npix, pattern) {  
  // パターンを流す
```

```
    for (let i = 0; i < 30; i++) {  
      const p = i % pattern.length;  
      const spattern = [];  
      for (let px = 0; px < pattern.length; px++) {  
        const pp = (i + px) % pattern.length;  
        spattern.push(pattern[pp]);  
      }  
      console.log(spattern);  
      await setPattern(npix, spattern);  
      await sleep(200);  
    }  
  }  
}
```

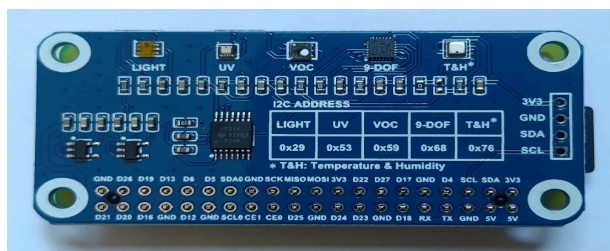
電子ペーパー

WAVESHARE20471複合センサーボード

概要

- 多機能環境センサーボードで、複数のセンサーを1枚の基板に統合
- I2C 通信を介して接続し、温度、湿度、気圧、照度、紫外線、揮発性有機化合物 (VOC)、および 9 軸モーション（加速度、ジャイロ、磁気）など、さまざまな環境データを取得可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/bme280
npm i @chirimen/icm20948
npm i @chirimen/ltr390
npm i @chirimen/tsl2591
npm i @chirimen/sgp40
```

サンプルコード (main.js)

```
// webI2C
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";

// I2c device drivers
import BME280 from "@chirimen/bme280"; // BME280
import ICM20948 from "@chirimen/icm20948"; // ICM20948
import LTR390 from "@chirimen/ltr390"; // LTR390
import TSL2591 from "@chirimen/tsl2591"; // TSL2591
import SGP40 from "@chirimen/sgp40"; // SGP40

const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

var sgp40, tsl2591, ltr390, icm20948, bme280;

main();

async function main() {
  // initialize I2C
  const i2cAccess = await requestI2CAccess();
  const ic2Port = i2cAccess.ports.get(1);
  // initialize I2C devices

  sgp40 = new SGP40(ic2Port);
  tsl2591 = new TSL2591(ic2Port);
  ltr390 = new LTR390(ic2Port);
  icm20948 = new ICM20948(ic2Port);
  bme280 = new BME280(ic2Port, 0x76); // これだけ0x76忘れず
  に

  await sgp40.init();
```

```

    await tsl2591.init();
    await ltr390.init();
    await icm20948.init();
    await bme280.init();

    // sensing loop
    while (true) {
        const sgp = await sgp40.measureRaw(25, 50);
        const tsl = await tsl2591.Lux();
        const ltr = await ltr390.UVS();
        const icm = await icm20948.getdata();
        const bme = await bme280.readData();

        console.log("=====
=====");
        console.log("Pressure: ",bme.pressure);
        console.log("Temperature: ",bme.temperature);
        console.log("Humidity: ",bme.humidity);
        console.log("Lux: ",tsl);
        console.log("UV: ",ltr);
        console.log("Gas: ", sgp );
        console.log(`Roll = ${icm[0].toFixed(2)} , Pitch = ${icm[1].toFixed(2)} , Yaw = ${icm[2].toFixed(2)}`);
        console.log(`Acceleration: X = ${icm[3]}, Y = ${icm[4]}, Z = ${icm[5]}`);
        console.log(`Gyroscope:      X = ${icm[6]} , Y = ${icm[7]} , Z = ${icm[8]}`);
        console.log(`Magnetic:      X = ${icm[9]} , Y = ${icm[10]} , Z = ${icm[11]}`);

        await sleep(1000);
    }
}

```


特記事項

説明ページの組み立て方^{*1}に従って Raspberry Pi Zero に接続したら完了です。

1. https://www.waveshare.com/wiki/Environment_Sensor_HAT#Hardware_connection

モーター制御

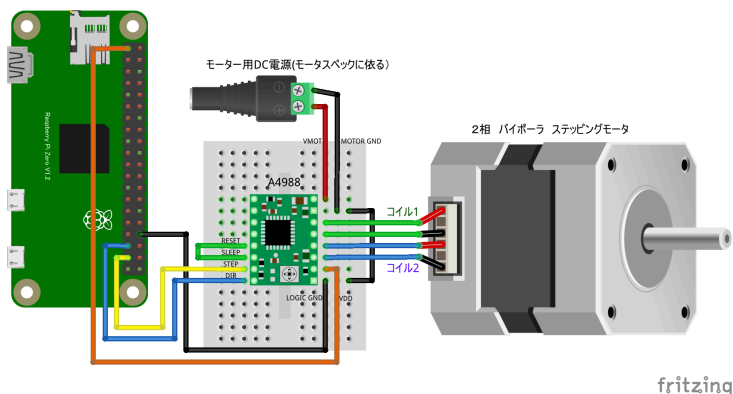
ステッピングモータードライバ

A4988ドライバによるステッピングモータ制御

概要

- I²C 接続モータードライバ
- 電圧設定と回転方向を指定するだけでモーターを簡単に制御可能

配線図



CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js";  
const sleep = msec => new Promise(resolve => setTimeout(reso
```

```
lve, msec));

let portA, portB;

async function init() {
  const gpioAccess = await requestGPIOAccess();
  portA = gpioAccess.ports.get(26);
  await portA.export("out");
  await portA.write(0);
  portB = gpioAccess.ports.get(19);
  await portB.export("out");
  await portB.write(0);
}

async function main() {
  await init();
  let direction = 1;
  for (; ;) {
    console.log("Move:", direction);
    await stepMove(100, direction);
    await sleep(500);
    direction = direction == 1 ? 0 : 1;
  }
}

async function stepMove(steps, direction) {
  await portB.write(direction);
  for (let i = 0; i < steps; i++) {
    await portA.write(1);
    await sleep(1);
    await portA.write(0);
    await sleep(20);
  }
}
```

```
}
```

```
main();
```

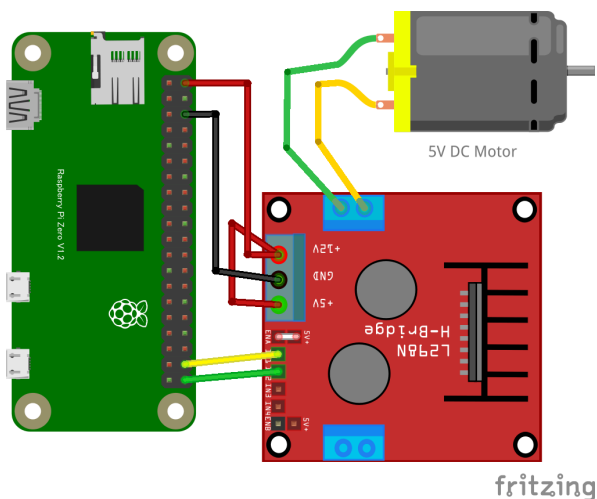
H ブリッジ

HBridge1

概要

- H ブリッジモータードライバは正転 [1,0]・逆転 [0,1]・ブレーキ [1,1]・フリー [0,0]の 4 状態を GPIO の 2 つ の信号線を使って指示

配線図



CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
// Hブリッジモータードライバは正転[1,0]・逆転[0,1]・ブレーキ[1,1]・
// フリー[0,0]の4状態を
// GPIOの2つの信号線を使って指示します
```

```
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

const portAddrs = [20, 21]; // HブリッジコントローラをつなぐGPIOポート番号
let ports;

main();

async function main() {
  await init();

  for (; ;) {
    console.log("fwd");
    await fwd();
    await sleep(1000);
    console.log("rev");
    await rev();
    await sleep(1000);
    console.log("brake");
    await brake();
    await sleep(1000);
  }
}

async function init() {
  // ポートを初期化するための非同期関数
  const gpioAccess = await requestGPIOAccess(); // thenの前の関数をawait接頭辞をつけて呼び出します。
  ports = [];

  for (let i = 0; i < 2; i++) {
    ports[i] = gpioAccess.ports.get(portAddrs[i]);
  }
}
```

```
    await ports[i].export("out");
  }
  for (let i = 0; i < 2; i++) {
    ports[i].write(0);
  }
}

async function free() {
  ports[0].write(0);
  ports[1].write(0);
}

async function brake() {
  ports[0].write(1);
  ports[1].write(1);
  await sleep(300); // 300ms待機してフリー状態にします
  ports[0].write(0);
  ports[1].write(0);
}

async function fwd() {
  ports[0].write(1);
  ports[1].write(0);
}

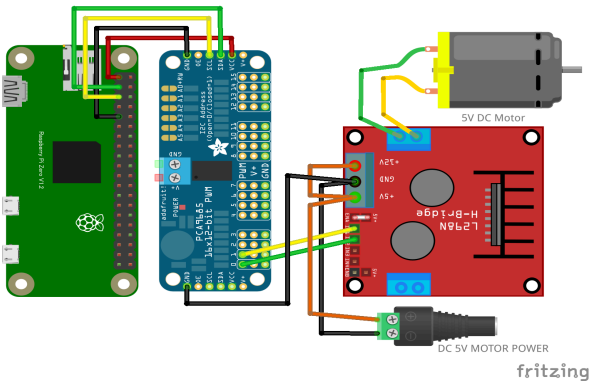
async function rev() {
  ports[0].write(0);
  ports[1].write(1);
}
```

PCA9685 16 チャンネル PWM ドライバ

概要

- PCA9685 16 チャンネルPWM ドライバーと HBridge モータードライバーで、DC モーターを正転・逆転・速度コントロール

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/pca9685-pwm
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import PCA9685_PWM from "@chirimen/pca9685-pwm";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

let pca9685pwm;
```



```

main();

async function main() {
  await init();

  let direction = -1;
  for (; ;) {
    direction = (direction == 1) ? -1 : 1; // 三項演算子で
directionを反転
    console.log("direction:", direction);
    console.log("speedUp");
    for (let speed = 0; speed <= 1; speed += 0.1) {
      await setMotor(direction, speed);
      await sleep(200);
    }
    await sleep(500);
    console.log("speedDown");
    for (let speed = 1; speed >= 0; speed -= 0.1) {
      await setMotor(direction, speed);
      await sleep(200);
    }
    console.log("stop");
    await setMotor(0, 0);
    await sleep(1000);
  }
}

async function init() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  pca9685pwm = new PCA9685_PWM(port, 0x40);
  await pca9685pwm.init(100);
}

async function setMotor(direction, speed) {

```

```
// direction: モーターの回転方向 正転:1,逆転:-1,停止:0
// speed: モーターのスピード 0:停止...1.0:全速
if (direction == 1) {
    await pca9685pwm.setPWM(1, 0);
    await pca9685pwm.setPWM(0, speed);
} else if (direction == -1) {
    await pca9685pwm.setPWM(0, 0);
    await pca9685pwm.setPWM(1, speed);
} else {
    await pca9685pwm.setPWM(0, 0);
    await pca9685pwm.setPWM(1, 0);
}
}
```

特記事項

- HBridge コントローラは L298N 以外にも L9110S、MX1508 等も同様に使用できます。
- ただし、これらのドライバでは +5V と +12V 端子の代わりに、一個の VCC や電源 (+) 端子だけのものが多いので、モーター電源はそこに接続

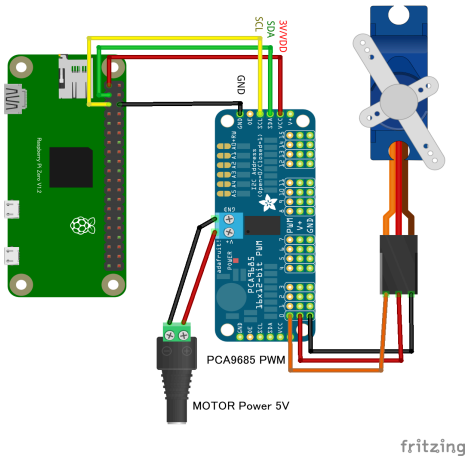
PWM コントローラー

PCA9685 16 チャンネルサーボモーター PWM ドライバー

概要

- PWM 制御 IC で、最大 16 個のサーボモーターや LED を個別に制御可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/pca9685
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import PCA9685 from "@chirimen/pca9685";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const pca9685 = new PCA9685(port, 0x40);
  // servo setting for sg90
  // Servo PWM pulse: min=0.0011[sec], max=0.0019[sec] angle
  =+-60[deg]
  await pca9685.init(0.001, 0.002, 30);
  for (;;) {
    await pca9685.setServo(0, -30);
    console.log("-30 deg");
    await sleep(1000);
    await pca9685.setServo(0, 30);
    console.log("30 deg");
    await sleep(1000);
  }
}
```

アナログ・デジタル変換

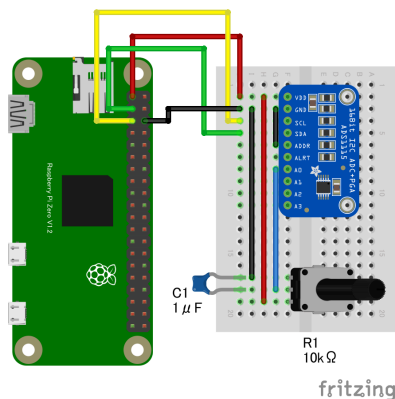
ADC

ADS1015 12 ビット AD コンバータ

概要

- I²C インターフェースを介してマイコンと接続し、アナログ信号を高精度にデジタル化することが可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/ads1015
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/
index.js";
import ADS1015 from "@chirimen/ads1015";
const sleep = msec => new Promise(resolve => setTimeout(reso
lve, msec));

main();

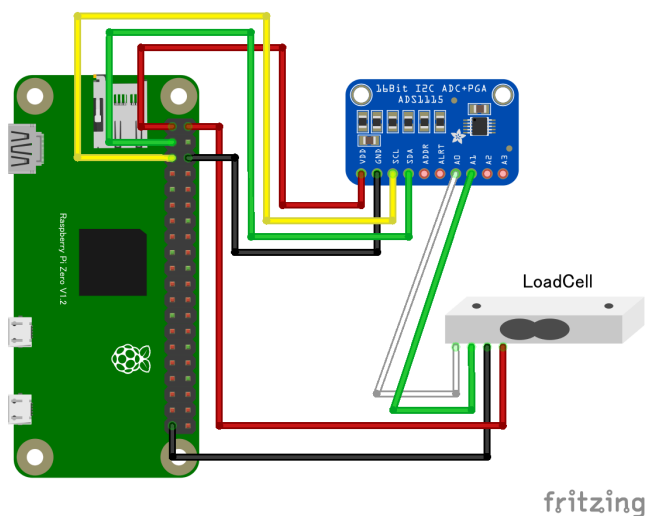
async function main() {
  var i2cAccess = await requestI2CAccess();
  var port = i2cAccess.ports.get(1);
  var ads1015 = new ADS1015(port, 0x48);
  await ads1015.init();
  for (;;) {
    try {
      var value = await ads1015.read(0);
      console.log("value:", value);
    } catch (error) {
      console.error("error: code:" + error.code + " message:
" + error.message);
    }
    await sleep(100);
  }
}
```

ADS1115 16bit ADC 差動入力によるロードセルの使用

概要

- 16 ビットの高精度アナログ-デジタルコンバータ (ADC)
- I²C インターフェースを介してマイコンと接続し、アナログ信号をデジタル化することが可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/ads1x15
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";  
import ADS1X15 from "@chirimen/ads1x15";
```

```

const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const ads1115 = new ADS1X15(port, 0x48);
  // If you uses ADS1115, you have to select "true", otherwise select "false".
  await ads1115.init(true, 7); // High Gain
  console.log("init complete");
  var firstTime = true;
  var tare;
  while (true) {
    var difA = await ads1115.read("0,1"); // p0-p1 differential mode
    console.log("dif chA(0-1):" + difA.toString(16) + " : " + ads1115.getVoltage(difA).toFixed(6) + "V");
    if (firstTime) {
      tare = difA;
      firstTime = false;
    }
    var weight = difA - tare;
    console.log("rawData - Tare:" + weight.toString(16) + " : " + ads1115.getVoltage(weight).toFixed(6) + "V");
    await sleep(500);
  }
}

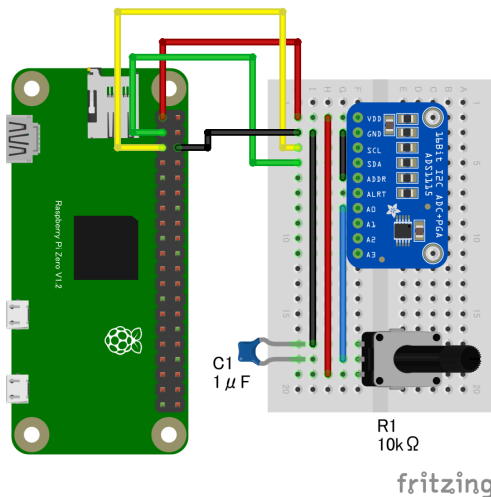
```


ADS1115 16bit ADC

概要

- 16 ビットの高精度アナログ-デジタルコンバータ (ADC)
- I²C インターフェースを介してマイコンと接続し、アナログ信号をデジタル化することが可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/ads1x15
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import ADS1X15 from "@chirimen/ads1x15";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const ads1x15 = new ADS1X15(port, 0x48);
  // If you uses ADS1115, you have to select "true", otherwise select "false".
  await ads1x15.init(true);
  console.log("init complete");
  while (true) {
    let output = "";
    // ADS1115 has 4 channels.
    for (let channel = 0; channel < 4; channel++) {
      const rawData = await ads1x15.read(channel);
      const voltage = ads1x15.getVoltage(rawData);
      output += `CH${channel}:${voltage.toFixed(3)}V `;
    }
    console.log(output);

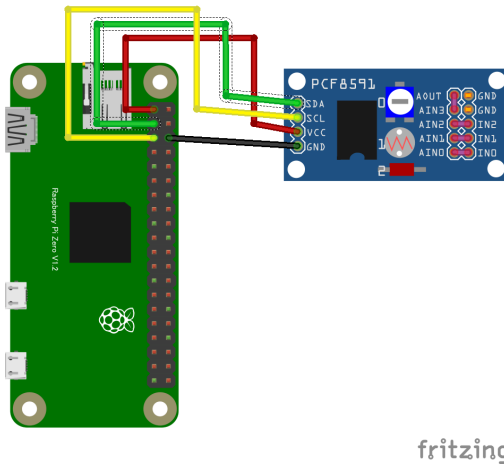
    await sleep(500);
  }
}
```

PCF8591 8bit AD,DA コンバーター

概要

- I²C インターフェース対応 8 ビット A/D or D/A コンバータ IC
- この IC は、4 つのアナログ入力チャンネルと 1 つのアナログ出力チャンネルを装備
- マイコンボード（Arduino、Raspberry Pi など）と組み合わせて、アナログ信号の読み取りや出力に利用可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/pcf8591
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/
index.js";
import PCF8591 from "@chirimen/pcf8591";
const sleep = msec => new Promise(resolve => setTimeout(reso
lve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const pcf8591 = new PCF8591(port, 0x48);
  await pcf8591.init();

  // Set DAC voltage (Range: 0V - 3.3V)
  await pcf8591.setDAC(3.3);

  while (true) {
    let output = "";

    // PCF8591 has 4 channels
    for (let channel = 0; channel < 4; channel++) {
      const voltage = await pcf8591.readADC(channel);
      output += `CH${channel}:${voltage.toFixed(3)}V `;
    }
    console.log(output);

    await sleep(500);
  }
}
```

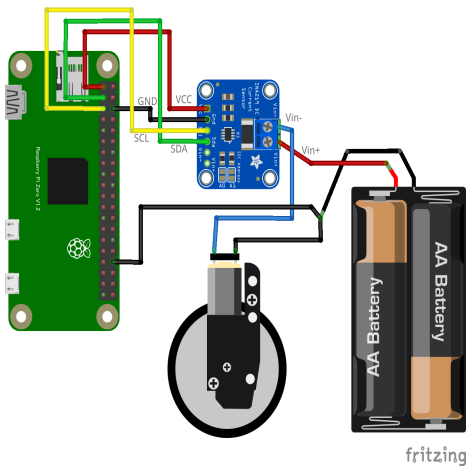
電流センサー

INA219 電流センサー

概要

- 高精度なデジタル電流・電圧・電力センサー
- I²C インターフェースを介してマイコンと通信し、シャント抵抗を用いて電流を測定

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/ina219
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/
index.js";
import INA219 from "@chirimen/ina219";
const sleep = msec => new Promise(resolve => setTimeout(reso
lve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const ina219 = new INA219(port, 0x40);
  await ina219.init();
  await ina219.configure();

  while (true) {
    const voltage = await ina219.voltage();
    const supplyVoltage = await ina219.supply_voltage();
    const current = await ina219.current();
    const power = await ina219.power();
    const shuntVoltage = await ina219.shunt_voltage();

    console.log(
      [
        `Voltage: ${voltage.toFixed(3)}V`,
        `Supply voltage: ${supplyVoltage.toFixed(3)}V`,
        `Current: ${current.toFixed(2)}mA`,
        `Power: ${power.toFixed(2)}mW`,
        `Shunt voltage: ${shuntVoltage.toFixed(2)}mV`
      ].join(", ")
    );
  }
}
```

```
    await sleep(500);  
  }  
}
```

オーディオ

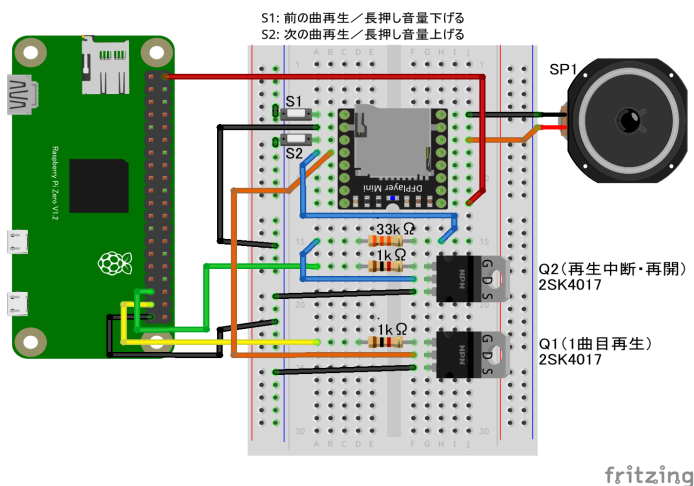
オーディオプレイヤー

DFPlayer Mini

概要

- MP3 プレーヤーボードを GPIO OUTPUT で制御

配線図



CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

let play1port;
let pauseport;

async function start_play() {
  const gpioAccess = await requestGPIOAccess();
  play1port = gpioAccess.ports.get(26); //
  pauseport = gpioAccess.ports.get(19);

  await play1port.export("out");
  await pauseport.export("out");

  await repeat_play();
}

async function repeat_play() {
  while (true) {
    // 1曲目を再生開始
    console.log("start play1")
    await play1port.write(1);
    await sleep(300); // 0.3秒保持して元に戻す
    await play1port.write(0);

    // 10秒保持
    await sleep(10 * 1000);

    // 一時停止
    console.log("stop")
  }
}
```

```
    await pauseport.write(1);  
    await sleep(300);  
    await pauseport.write(0);  
  
    // 3秒間一時停止  
    await sleep(3 * 1000);  
  }  
}  
  
start_play();
```

特記事項

- DFPlayer Mini ボードの ADKEY1 端子を Nch MOSFET を介して GPIO で制御します
 - 他にシリアル通信での制御も可能なボードですがこのサンプルは GPIO で制御できる ADKEY 端子を使用
- 電源投入後、ボリュームが最大になるので S1 スイッチは付けておきましょう
- ADKEY1/2 端子と抵抗を組み合わせることのできるいろいろなコントロールが可能です。

参考

GPIO 端子の使用数を増やすと制御できる種類も増やせます。

- [参考ページ](#)^{*1}
- [メーカーサイトの説明ページ](#)^{*2}

1. <https://chitakekoubou.blogspot.com/p/dfplayeradkeyio.html>
2. https://wiki.dfrobot.com/DFPlayer_Mini_SKU_DFR0299

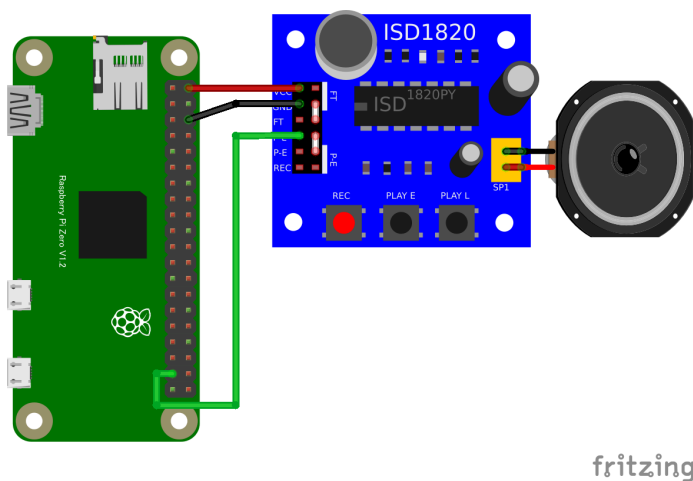
音声録音

ISD1820ボイスレコーダ (GPIO OUTPUT)

概要

- 音声の録音と再生を簡単にできるモジュール

配線図



CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
async function blink() {  
  const gpioAccess = await requestGPIOAccess();  
  const port = gpioAccess.ports.get(26);  
  
  await port.export("out");  
  
  for (;;) {  
    await port.write(1);  
    await sleep(3000); // 3秒間再生  
    await port.write(0);  
    await sleep(1000); // 1秒間停止  
  }  
}  
  
blink();
```

特記事項

- GPIO ポート 26 を ISD1820 の P-L 端子に接続します。(P-L 端子が High になっている間だけ再生します(P-L ボタンと同じ動作))
- P-E 端子に繋いだ場合、録音した内容を全部再生するようになります。(PLAY E ボタンと同じ動作)
- 録音は REC ボタンを押している間録音できます(最大 10 秒)
- REC 端子を GPIO に接続すると、録音をコンピュータで制御することもできます。(REC 端子が High になっている間だけ録音)

その他

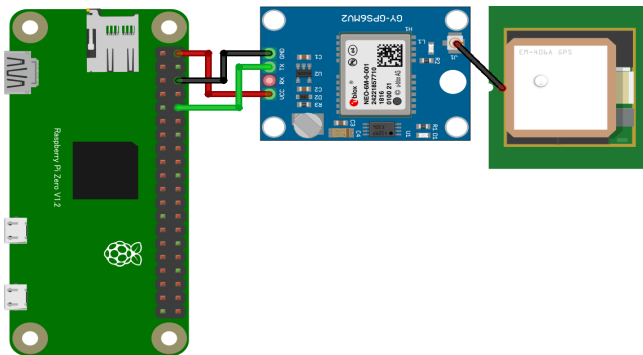
GPS

Serial GPS

概要

- 高性能 GPS 受信チップ NEO-6M を搭載した GPS モジュール
- Pi Zero のシリアル端子に、GY-GPS6MV2 等の GPS レシーバ基板を繋いで使用

配線図



fritzing

CHIRIMEN 用ドライバのインストール

- 不要

サンプルコード (main.js)

```
// シリアルからGPSデータ受信
import { SerialPort } from 'serialport';
import { ReadlineParser } from '@serialport/parser-readline'
```

```

;
import GPSPackage from 'gps';
const GPS = GPSPackage.GPS;

const port = new SerialPort({ path: '/dev/ttyS0', baudRate:
9600 })
const parser = port.pipe(new ReadlineParser())
const gps = new GPS();

parser.on('data', function (txtData) {
  gps.update(txtData);
});

gps.on('data', function (data) {
  if (data.type == "GGA") { // "RMC"タイプデータを読むと速度(
ノット)が得られる
    console.log(data);
  }
  /**
  console.log("data:", data);
  console.log("stat:", gps.state);
  console.log("=====
=====");
  **/
});

```

特記事項

- OSの設定
 - `sudo raspi-config`
 - Interface Options -> Serial Port -> Login over serial: いいえ , serial port enabled: はい -> Finish (reboot)
 - Note: この設定はUSBシリアルコンソールログインには影響しない
- 結線 (GPSのRX端子の結線は基本動作では不要): 下図参照

- 動作検証したモジュール (GY-NEO6MV2, 基板の印刷は GY-GPS6MV2)
 - <https://electronicwork.shop/items/625c1ca99fe3d707d725cbe1>^{*1}
- 同等品と考えられるもの
 - <https://www.amazon.co.jp/dp/B07LF6KGR8>^{*2}
 - <https://www.aitendo.com/product/10255>^{*3}
- 動作確認
 - `cat /dev/ttyS0`
 - Note: GPS 衛星電波受信されていなくてもメッセージが出力される。
`/dev/serial0` も使える。測位成功するとメッセージが派手になり、LED が点滅する (LED は測位成功していないときは消灯)
- シリアルポート及びGPSのライブラリを導入(myAppディレクトリで)
 - `cd myApp`
 - `npm install serialport gps`
- アプリの実行
 - `main.js` を myApp 下に保存
 - `node main.js`

1. <https://electronicwork.shop/items/625c1ca99fe3d707d725cbe1>
 2. <https://www.amazon.co.jp/dp/B07LF6KGR8>
 3. <https://www.aitendo.com/product/10255>

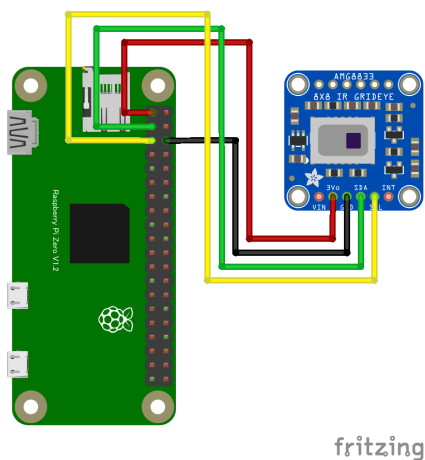
サーマルカメラ

AMG8833 サーモグラフィー

概要

- 赤外線アレイセンサー Grid-EYE シリーズの高性能モデル
- 8×8（64 ピクセル）の熱センサーを搭載し、非接触で温度分布を 2 次元的に測定可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/amg8833
```


サンプルコード (main.js)

```
import {requestI2CAccess} from "./node_modules/node-web-i2c/
index.js";
import AMG8833 from "@chirimen/amg8833";
const sleep = msec => new Promise(resolve => setTimeout(reso
lve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const amg8833 = new AMG8833(port, 0x69);
  await amg8833.init();

  while (true) {
    const data = await amg8833.readData();
    for (const row of data) {
      // degree Celsius
      console.log(row.map(value => value.toFixed(2)).join("
"));
    }

    console.log();
    await sleep(500);
  }
}
```

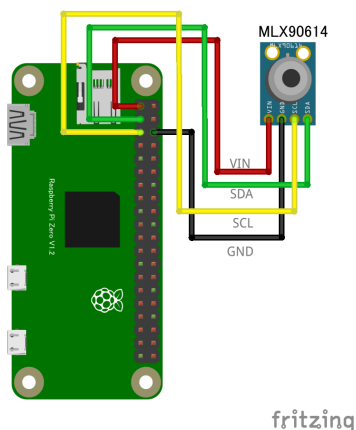
赤外線温度センサー

MLX90614 赤外線温度センサー

概要

- 高精度な非接触型赤外線温度センサー
- 赤外線サーモパイル検出器と信号処理用の ASIC を一体化した TO-39 パッケージに収められており、物体や環境の温度を非接触で測定可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/mlx90614
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "./node_modules/node-web-i2c/index.js";  
import MLX90614 from "@chirimen/mlx90614";
```

```

const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const mlx90614 = new MLX90614(port, 0x5a);
  await mlx90614.init();

  while (true) {
    // Temperature of object that the sensor looking at. (Measured by IR sensor)
    const objectTemperature = await mlx90614.get_obj_temp();
    // Temperature measured by the chip. (The package temperature)
    const ambientTemperature = await mlx90614.get_amb_temp();
    ;
    console.log(
      [
        `Object temperature: ${objectTemperature.toFixed(2)} degree`,
        `Ambient temperature: ${ambientTemperature.toFixed(2)} degree`
      ].join(", ")
    );

    await sleep(500);
  }
}

```

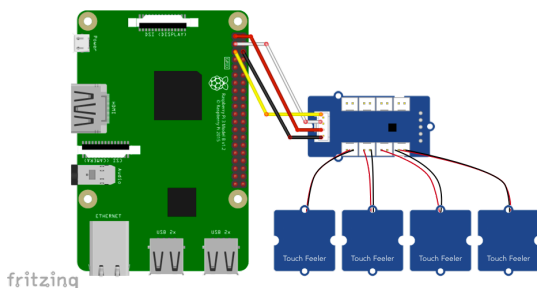
タッチセンサー

MPR121 静電容量センサ(12ch)

概要

- 12 チャンネルの静電容量式タッチセンサーコントローラ
- このセンサーは、非接触でのタッチ検出や近接検出が可能

配線図



CHIRIMEN 用ドライバのインストール

```
npm i @chirimen/grove-touch
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "./node_modules/node-web-i2c/  
index.js";  
import GroveTouch from "@chirimen/grove-touch";  
const sleep = msec => new Promise(resolve => setTimeout(reso  
lve, msec));  
  
main();
```

```
async function main() {  
  var i2cAccess = await requestI2CAccess();  
  var port = i2cAccess.ports.get(1);  
  var touchSensor = new GroveTouch(port, 0x5a);  
  await touchSensor.init();  
  for (;;) {  
    var ch = await touchSensor.read();  
    console.log(JSON.stringify(ch));  
    await sleep(100);  
  }  
}
```



Special Thanks

表紙・イラスト

- @天灯せいた^{*1}

Works (alphabet order)

- @elie-j^{*2}
- @gurezo^{*3}
- @happysato^{*4}
- @satakagi^{*5}

1. <https://www.pixiv.net/users/31028725>

2. <https://github.com/elie-j>

3. <https://github.com/gurezo>

4. <https://github.com/happysato>

5. <https://github.com/satakagi>